rump

A Reference Manual for the Multiprecision Arithmetic Crate

version 0.4.0

Abstract

rump is a dependency-free Rust crate of multiprecision integer arithmetic implemented from the literature: Knuth's Algorithm D for division, Montgomery and Barrett modular reduction, Karatsuba, Toom–Cook, and exact NTT multiplication, Newton's reciprocal for wide division, Lehmer's gcd with subquadratic Half-GCD, the Jacobi symbol from the Euclidean quotient sequence, Tonelli–Shanks and Cipolla square roots, Miller–Rabin and Baillie–PSW primality, binary fields $\mathbb{F}_{2^m}$, univariate polynomials over $\mathbb{Z}$ and $\mathbb{F}_p$ with factorization, and integral LLL lattice reduction. This manual documents every public primitive: the mathematics that defines it, the algorithm that computes it, and a worked example of its use. Code examples are pinned by the crate's test suite.

Contents

1 Preliminaries **2**
- 1.1 Scope and intended use . . . 2
- 1.2 Imports and naming . . . 3
- 1.3 Notation . . . 3

2 Unsigned integers: `BigUint` **4**
- 2.1 Construction, bytes, and radix strings . . . 4
- 2.2 Comparison and predicates . . . 4
- 2.3 Addition and subtraction . . . 5
- 2.4 Multiplication . . . 5
- 2.5 Squaring . . . 7
- 2.6 Division and remainder . . . 7
- 2.7 Shifts and bit access . . . 8
- 2.8 Integer roots . . . 8
- 2.9 Size estimates . . . 8
- 2.10 Division by an invariant divisor . . . 9

3 Signed integers: `BigInt` and `Sign` **9**

4 Barrett reduction: `BarrettContext` **10**

5 The Montgomery domain: `MontgomeryContext` **11**

6 Binary fields: `Gf2m` 13
6.1 Arithmetic . . . 13
6.2 Trace, half-trace, square roots, and quadratics . . . 14
6.3 Irreducibility: Rabin's test . . . 14

7 Number theory 15
7.1 Greatest common divisors . . . 15
7.2 Quadratic-residue symbols . . . 16
7.3 Modular exponentiation and inverses . . . 16
7.4 Square roots rem a prime . . . 17
7.5 Square roots rem a prime power . . . 18
7.6 The Chinese Remainder Theorem . . . 19
7.7 Valuation . . . 20
7.8 Rational reconstruction . . . 21
7.9 Product trees, remainder trees, and batch smoothness . . . 21
7.10 Primality . . . 22

8 Polynomials: `PolyZ` and `PolyMod` 25
8.1 Arithmetic over $\mathbb{Z}$ . . . 25
8.2 Division over $\mathbb{Z}$ . . . 25
8.3 Resultants and discriminants . . . 26
8.4 Arithmetic over $\mathbb{F}_p$ . . . 26
8.5 Factorization over $\mathbb{F}_p$ . . . 27
8.6 Quotient rings, lifting, and base expansions . . . 28
8.7 Real roots . . . 30

9 Lattice reduction: `lll_reduce` 31
9.1 Two dimensions under a diagonal form . . . 32

10 Linear algebra over a prime field: `gfp` 33

11 Random sampling 34

12 Panics 36

1 Preliminaries

1.1 Scope and intended use

Two properties define the intended use of every primitive in this manual.

Contract 1.1 (Variable time)**.** *Operations take data-dependent paths and their running time may leak their operands. Do not use rump where timing must not leak secrets.*

Contract 1.2 (Non-secret data)**.** *rump is not a secret-scrubbing or constant-time library. Nothing is wiped: values live in ordinary heap buffers, freed memory keeps its contents, and* `Debug` *prints every limb. Cryptographic memory hygiene and constant-time operation are out of scope, left to the consumer that handles key material.*

The crate is portable and carries no target-width restriction. Every place that turns a limb count into a bit index goes through one checked multiplication, so an operand too wide to index on the target refuses rather than wrapping; on a 32-bit `usize` that boundary is reachable, at operands of roughly 537 MB for len · 64 and 268 MB for len · 128, and on a 64-bit one it is not.

1.2 Imports and naming

The crates.io package is `rust-mp`; the library target is `rump`, so code always says `use rump::...`:

```
[dependencies]
rust-mp = "0.3"
```

```
use rump::finite_field::Gf2m;
use rump::gfp::{kernel_vector, minimal_polynomial, Field, Kernel, SparseMatrix};
use rump::integer::WordReciprocal;
use rump::lattice::{gauss_reduce_weighted, lll_reduce, lll_reduce_delta,
                    ReductionError};
use rump::modular::{
    mod_inverse, mod_inverse_batch, mod_inverse_u128, mod_inverse_u64, mod_pow,
    mod_sqrt,
    mod_sqrt_prime_power, BarrettContext, MontgomeryContext, MontgomeryScratch,
};
use rump::number_theory::{
    crt_combine, crt_combine_balanced, crt_combine_u64, gcd, gcd_extended, gcd_u64,
    is_lucas_probable_prime, is_prime_aks, is_probable_prime, is_probable_prime_bpsw,

    is_strong_lucas_probable_prime,
    jacobi, kronecker, lcm, legendre, miller_rabin_with_bases, miller_rabin_witness,
    primes_below, product_tree, rational_reconstruct, rational_reconstruct_bounded,
    remainder_tree, remove_factor, smooth_parts, valuation, CrtBasis, SmoothnessBase,

};
use rump::parallelism::{budget, gather_costs, with_budget};
use rump::polynomial::{PolyMod, PolyZ, RealRootError};
use rump::random::{
    random_below, random_coprime_below, random_nonzero_below, random_probable_prime,
     RandomSource,
};
use rump::{BigInt, BigUint, Sign};
use core::num::NonZeroU64;
```

This manual documents the crate at version 0.4.0. Where a threshold is named in `typewriter` capitals it is a constant in `src/`, and the constant is the authority for its value.

1.3 Notation

Throughout, $\beta = 2^{64}$ is the limb base. A non-negative integer a is represented as a little-endian limb vector

$$a = \sum_{i=0}^{k-1} a_i \, \beta^i, \qquad 0 \le a_i < \beta, \tag{1}$$

normalized so the most significant limb is non-zero (zero is the empty vector). $\lfloor x \rfloor$ is the floor, $\lg x = \log_2 x$, and $\left(\frac{a}{n}\right)$ is the Jacobi (or, over a prime, Legendre) symbol. Polynomial rings are written $\mathbb{Z}[x]$ and $\mathbb{F}_p[x]$; $\mathrm{lc}(f)$ is the leading coefficient of f and $\deg f$ its degree, with $\deg 0 = -\infty$.

2 Unsigned integers: `BigUint`

2.1 Construction, bytes, and radix strings

`zero`, `one`, `from_u64`, and `from_u128` build small values. Bytes come in both orders: `from_be_bytes` / `to_be_bytes` are big-endian, as DER, PEM, and most RFC wire formats use, and `from_le_bytes` / `to_le_bytes` are their little-endian mirror, as the X25519 and Ed25519 encodings use. Decoding the empty slice gives zero, and zero bytes at the high end are ignored. The encoders emit the minimal $\lceil b/8 \rceil$ bytes for a b-bit value — a single `0x00` for zero — while `to_be_bytes_padded(w)` and `to_le_bytes_padded(w)` zero-fill the high end to w bytes and panic when the value does not fit. Each encoder allocates its output once, at its final length, and writes the limbs straight into it, so it leaves no other heap copy of the value's bytes. `low_u128` and `low_bits(k)` ($= a \bmod 2^k$) read the low end directly.

`from_str_radix` / `to_str_radix` convert against any radix $2 \le r \le 36$; `Display` and `FromStr` are the base-10 special case, and `FromStr`'s error type is the exported `ParseBigIntError`. Power-of-two radices convert by direct bit packing. The rest run classical word-sized conversion at small sizes and switch to divide-and-conquer against a ladder of squared radix powers: parsing from `RADIX_FROM_DC_THRESHOLD_DIGITS` (1024 digits) and rendering from `RADIX_TO_DC_THRESHOLD_BITS` (2048 bits), each recursion converting classically once a subproblem falls below its floor (`RADIX_FROM_DC_BASE_DIGITS`, `RADIX_TO_DC_BASE_BITS`, both 512). To parse n digits, split at the largest r^{2^j} below the value, giving the recurrence $T(n) = 2\,T(n/2) + M(n)$, where M is the multiplication cost — subquadratic wherever M is.

```
let n = BigUint::from_str_radix("deadbeef", 16).expect("valid hex");
assert_eq!(n, BigUint::from_u64(0xdead_beef));
assert_eq!(n.to_str_radix(16), "deadbeef");
assert_eq!(n.to_string(), "3735928559");
assert_eq!("3735928559".parse::<BigUint>(), Ok(n));

let value = BigUint::from_be_bytes(&[0x01, 0x00]);
assert_eq!(value, BigUint::from_u64(256));
assert_eq!(value.to_be_bytes_padded(4), vec![0x00, 0x00, 0x01, 0x00]);
assert_eq!(BigUint::from_le_bytes(&[0x00, 0x01]), value);
assert_eq!(value.to_le_bytes(), vec![0x00, 0x01]);
assert_eq!(value.to_le_bytes_padded(4), vec![0x00, 0x01, 0x00, 0x00]);
```

2.2 Comparison and predicates

`BigUint` implements `Ord`: comparison walks limbs from the top and stops at the first difference. `bits` returns the significant bit count, $\lfloor \lg a \rfloor + 1$ for $a > 0$ and 0 for $a = 0$; `is_zero`, `is_one`, and `is_odd` are constant-limb predicates; `popcount` is the Hamming weight; `trailing_zeros` is the 2-adic valuation $v_2(a)$, `None` at zero.

2.3 Addition and subtraction

`add` / `sub` return new values; `+=` / `-=` on a borrowed right-hand side work in place; `add_into` / `sub_into` are three-operand forms whose output buffer is reused across calls (the shape of GMP's `mpz_add`), and `Clone::clone_from` copies a value into existing storage the same way. Both directions are the classical limb-wise carry and borrow chains. Subtraction panics on underflow: the type is unsigned, and a result that would be negative is a programming error — use `BigInt` where signs can go negative.

```
let a = BigUint::from_u64(1_000);
let b = BigUint::from_u64(37);
assert_eq!(a.add(&b), BigUint::from_u64(1_037));
assert_eq!(a.sub(&b), BigUint::from_u64(963));

// Three-operand form: `out`'s storage is reused across calls.
let mut out = BigUint::zero();
out.add_into(&a, &b);
assert_eq!(out, BigUint::from_u64(1_037));
out.sub_into(&a, &b);
assert_eq!(out, BigUint::from_u64(963));
out.mul_into(&a, &b);
assert_eq!(out, BigUint::from_u64(37_000));

// A fold modulo the Mersenne number 2^5 - 1 = 31, in place: the bits above
// the fifth weigh one, so they are added back.
let mut x = BigUint::from_u64(1_000); // 1000 = 31*32 + 8
let mut high = x.clone();
high.shr_bits(5);
x.keep_low_bits(5);
x += &high;
assert_eq!(x, BigUint::from_u64(8 + 31)); // one more fold would give 8
```

2.4 Multiplication

`mul` dispatches on operand size across a ladder of kernels, each asymptotically cheaper but with a larger constant, so each engages only past its measured crossover. `square` has its own ladder (§2.5), since a squaring can form each distinct cross term once. `mul_into` is the product's three-operand form: below `KARATSUBA_THRESHOLD_LIMBS` the schoolbook kernel runs straight into the output's buffer, so a caller that keeps its outputs alive multiplies without allocating; `keep_low_bits` is `self mod 2^k` in place, and with the in-place shifts and `+=` that is a Mersenne fold with no allocation per operation — a test counts the allocator's calls across a thousand such folds and requires zero.

Schoolbook (Knuth, Algorithm M). The base case, $O(mn)$ limb products, used below `KARATSUBA_THRESHOLD_LIMBS` (96 limbs).

Karatsuba (from `KARATSUBA_THRESHOLD_LIMBS`, 96 limbs). Admitted while long $< 2 \cdot$ short (`KARATSUBA_MAX_IMBALANCE`); at that ratio the shorter operand's high half is empty and the split degenerates to schoolbook. Split both operands at $B = \beta^s$, $a = a_1 B + a_0$, $b = b_1 B + b_0$. Three

half-width products replace four:

$$\begin{aligned} z_0 &= a_0 b_0, \qquad z_2 = a_1 b_1, \\ z_1 &= (a_0 + a_1)(b_0 + b_1) - z_0 - z_2 \;=\; a_0 b_1 + a_1 b_0, \\ ab &= z_2 B^2 + z_1 B + z_0, \end{aligned} \tag{2}$$

giving $T(n) = 3\,T(n/2) + O(n) = O(n^{\lg 3}) \approx O(n^{1.585})$.

Toom–Cook 3 and 4. Toom-3 engages from `TOOM3_THRESHOLD_LIMBS` (128 limbs) and Toom-4 from `TOOM4_THRESHOLD_LIMBS` (4096 limbs). Split each operand into k base-B digits and read it as a polynomial of degree $k-1$; the product polynomial has degree $2k-2$ and is determined by its values at $2k-1$ points. Toom-3 evaluates at $\{0, 1, -1, 2, \infty\}$ — five products of a third the size, $O(n^{\log_3 5}) \approx O(n^{1.465})$; Toom-4 at $\{0, 1, -1, 2, -2, 3, \infty\}$, seven points, $O(n^{\log_4 7}) \approx O(n^{1.404})$. Interpolation is exact: its divisions by small constants leave no remainder. Both kernels, and the NTT below, are balanced kernels: they accept a pair only while long $\le \frac{3}{2} \cdot$ short.

Exact two-prime NTT. At very large balanced sizes each limb is split into four base-2^{16} digits and the digit convolution is evaluated modulo two 31-bit NTT primes. Their product exceeds the asserted maximum coefficient through the supported 2^{26} transform length, so CRT recovery and the carry pass are exact; no floating-point rounding or probabilistic check is involved. Radix-2 stages run on scoped disjoint slices, use no more execution contexts than `available_parallelism` reports, and fall back to one if detection fails. Admission is by the shorter operand's width and the worker count the transform will get: `NTT_SERIAL_THRESHOLD_LIMBS` (131,072 limbs) on one context, `NTT_TWO_WORKER_THRESHOLD_LIMBS` (32,768) on two, and `NTT_PARALLEL_THRESHOLD_LIMBS` (8,192) on four or more. A radix-2 transform rounds the convolution up to a power of two, and that padding is not a reason to refuse: there is no gate on it.

Unbalanced operands (from `UNBALANCED_THRESHOLD_LIMBS`, 256 limbs). The balanced kernels all bound the length ratio of their operands. A lopsided product (long $\ge 2 \cdot$ short) whose shorter operand is past this kernel's own crossover — 256 limbs, far above Karatsuba's, because each block also pays dispatch and allocation overhead — is computed by block decomposition: with k the shorter length and $B = \beta^{\,k}$,

$$\text{long} \cdot \text{short} \;=\; \sum_i d_i \cdot \text{short} \cdot B^{\,i}, \qquad \text{long} = \sum_i d_i B^{\,i},\; 0 \le d_i < B, \tag{3}$$

a sum of balanced $k \times k$ products, each of which re-enters the dispatch and lands on Karatsuba or Toom, accumulated in place at its limb offset (shifting each product and adding at full width would cost $\Theta(\text{long}^2/k)$ limb copies and lose to flat schoolbook). Below the crossover the lopsided shapes stay schoolbook, which measurement favors there.

```
assert_eq!(a.mul(&b), BigUint::from_u64(37_000));
assert_eq!(b.square(), BigUint::from_u64(1_369));
```

2.5 Squaring

`square` is not `mul(a, a)`: a squaring forms each distinct cross term once and doubles it, so

$$a^2 \;=\; \sum_i a_i^2 \beta^{2i} \;+\; 2 \sum_{i<j} a_i a_j \beta^{\,i+j}, \tag{4}$$

which is $n(n+1)/2$ limb products against the general multiplication's n^2 — a ceiling of half the work, approached as the products come to dominate the carry walks. The kernel accumulates the strict upper triangle, doubles the whole buffer with one shift (which is what keeps the $2a_ia_j$ term from overflowing a `u128` accumulator), then adds the diagonal; the doubling cannot overflow, since twice the upper triangle is at most $a^2 < \beta^{2n}$.

Four regimes, each measured against the kernel it displaces. The general multiplication is used below `SQR_SCHOOLBOOK_MIN_LIMBS` (8 limbs), where the passes do not repay themselves. From there up to the Karatsuba crossover, `KARATSUBA_THRESHOLD_LIMBS`, the schoolbook squaring runs. From there up to `SQR_KARATSUBA_MAX_LIMBS` (1024 limbs) a Karatsuba squaring runs, whose three sub-products are themselves squares, since $z_1 = (a_0 + a_1)^2 - a_0^2 - a_1^2$. From 1024 limbs the general ladder's Toom kernels serve, and the NTT once it admits the width. NTT squaring uses one transform array, one forward transform per prime, and pointwise self-products rather than a general product's duplicate input transform, so it holds half the transform-array storage of a general product.

2.6 Division and remainder

`div_rem` returns (q, r) with $a = qd + r$, $0 \le r < d$, by Knuth's Algorithm D: normalize so the divisor's top limb is at least $\beta/2$, then estimate each quotient limb from the top two limbs of the running remainder against the top limb of the divisor,

$$\hat{q} \;=\; \min\!\left(\left\lfloor \frac{r_{j+n}\,\beta + r_{j+n-1}}{d_{n-1}} \right\rfloor,\; \beta - 1 \right), \tag{5}$$

which after the standard two-limb refinement is off by at most one, and the multiply-subtract step's borrow corrects that final case. Algorithm D is quadratic in the divisor's width; from `NEWTON_DIVISION_THRESHOLD_LIMBS` (3072 divisor limbs) the quotient is instead $\lfloor n \cdot \mu / \beta^{2k} \rfloor$ for the reciprocal $\mu = \lfloor \beta^{2k}/d \rfloor$ computed by Newton's iteration $x \mapsto x(2 - dx)$, which doubles its correct limbs at every step, $O(M(k))$ in all; the reciprocal is made exact by a final check $d\mu \le \beta^{2k} < d(\mu + 1)$ (Brent and Zimmermann, §4.2.2), and the quotient by a short correction. A one-limb divisor is Horner's recurrence in base β. `rem` keeps only the remainder; `div_rem_u64` and `rem_u64` divide by a machine word with no heap-allocated divisor. `mod_mul`, `mod_add`, `mod_sub`, and `mod_neg` are the one-shot modular operations: operands of any size, and a result in $[0, m)$. Negation returns 0 for a multiple of m and $m - (a \bmod m)$ otherwise. All panic on a zero divisor or modulus.

```
let n = BigUint::from_u64(1_000);
let d = BigUint::from_u64(7);
let (q, r) = n.div_rem(&d);
assert_eq!(q, BigUint::from_u64(142));
assert_eq!(r, BigUint::from_u64(6));
assert_eq!(n.rem_u64(7), 6);
```

```
// Negation lands in [0, m): -123 = 71 (mod 97), and 2 * 97 negates to zero.
let negated = BigUint::mod_neg(&BigUint::from_u64(123), &BigUint::from_u64(97));
assert_eq!(negated, BigUint::from_u64(71));
assert!(BigUint::mod_neg(&BigUint::from_u64(194), &BigUint::from_u64(97)).is_zero());
```

2.7 Shifts and bit access

`shl_bits` / `shr_bits` shift by any count ($a \cdot 2^n$ and $\lfloor a/2^n \rfloor$); `shl1` / `shr1` are the single-bit forms the division kernel uses in its hot loop. `bit(i)` tests bit i, `set_bit(i)` sets it, and `bitxor_assign` is bitwise XOR — field addition in $\mathbb{F}_{2^m}$ (§6).

2.8 Integer roots

`sqrt_rem` returns $(r, a - r^2)$ for $r = \lfloor \sqrt{a} \rfloor$; `sqrt_floor` is the root alone and skips the final squaring that produces the remainder. Both run Newton's iteration on integer division,

$$x_{k+1} \;=\; \left\lfloor \frac{x_k + \lfloor a/x_k \rfloor}{2} \right\rfloor, \qquad x_0 = 2^{\lceil b/2 \rceil} \geq \lceil \sqrt{a}\, \rceil, \quad b = \lfloor \lg a \rfloor + 1. \tag{6}$$

By the AM–GM inequality $\frac{1}{2}(x + a/x) \geq \sqrt{a}$ for every $x > 0$, so every iterate stays at or above $\lfloor \sqrt{a} \rfloor$; from a seed above the root the sequence decreases strictly until it reaches it, and the first non-decrease certifies the answer (Cohen, Algorithm 1.7.1). Convergence is quadratic: about $\lg \lg a$ iterations, each one division.

`nth_root_floor(k)` generalizes with the same certificate:

$$x_{j+1} \;=\; \left\lfloor \frac{(k-1)\, x_j + \lfloor a/x_j^{k-1} \rfloor}{k} \right\rfloor. \tag{7}$$

`is_square` answers by residue filters — squares occupy 44 of the 256 residues rem 256, and the single word modulus $9 \cdot 5 \cdot 7 \cdot 13 \cdot 17 = 69\,615$ folds five more character tests into one remainder — then one certified root for survivors. `is_perfect_power` tests $a = b^p$ for each prime $p \leq \lg a$ with one certified p-th root per exponent. `pow_u64` raises to a machine-word exponent by binary exponentiation.

```
let n = BigUint::from_u64(1_000_000);
let (root, rem) = n.sqrt_rem();
assert_eq!(root, BigUint::from_u64(1_000));
assert!(rem.is_zero());
assert!(n.is_square());
assert_eq!(n.nth_root_floor(3), BigUint::from_u64(100));
assert!(BigUint::from_u64(729).is_perfect_power()); // 3^6
```

2.9 Size estimates

`to_u64` narrows to a word when the value fits (`None` otherwise); `to_f64_lossy` saturates to infinity above the `f64` range; `ln_approx` computes $\ln a$ from the top bits and the bit length,

staying finite far past the `f64` range — for size-driven parameter heuristics, not for arithmetic. `digit_count(r)` returns the written length

$$d_r(a) \;=\; \lfloor \log_r a \rfloor + 1 \quad (a \geq 1), \qquad d_r(0) = 1, \tag{8}$$

from the limbs: a logarithm estimate with the power-of-r boundary — the one class the floating-point estimate cannot settle — corrected by comparison against r^{d-1} and r^d, so no expansion is produced. It panics for $r < 2$.

```
assert_eq!(BigUint::from_u64(1_000).digit_count(10), 4);
assert_eq!(BigUint::from_u64(999).digit_count(10), 3);
assert_eq!(BigUint::from_u64(255).digit_count(16), 2); // "ff"
assert_eq!(BigUint::zero().digit_count(10), 1); // written "0"
```

2.10 Division by an invariant divisor

`WordReciprocal::new(d)` precomputes, for a fixed non-zero d, the normalized divisor $d' = d \cdot 2^s$ with $s = \mathrm{clz}(d)$ and the word

$$v \;=\; \left\lfloor \frac{2^{128} - 1}{d'} \right\rfloor - 2^{64}, \tag{9}$$

which fits a word exactly because $d' \geq 2^{63}$ places the quotient in $[2^{64}, 2^{65})$. A later division by d then costs a multiplication by v and a correction instead of a hardware divide (Möller & Granlund, IEEE Transactions on Computers 60 (2011), Algorithm 4). The answers are those of `div_rem_u64` and `rem_u64`; only the cost moves. Construction panics for $d = 0$.

Dividing a multi-limb value by a word is Horner's recurrence in base 2^{64}, and each step of it is a two-word-by-one-word division, so `rem_reciprocal` and `div_rem_reciprocal` use the same kernel. Scaling both operands by 2^s leaves the quotient unchanged and multiplies the remainder by 2^s, so the remainder is shifted back and the quotient needs no correction.

`rem_euclid_i64` is the signed entry point, returning the non-negative residue in $[0, d)$ — what an index into a table requires, and what a truncating remainder does not give.

```
let r = WordReciprocal::new(NonZeroU64::new(1_000_003).expect("non-zero"));
assert_eq!(r.divisor(), 1_000_003);
assert_eq!(r.div_rem(2_000_007), (2, 1));
assert_eq!(r.rem(2_000_006), 0);

assert_eq!(r.rem_euclid_i64(-1), 1_000_002);
assert_eq!(r.rem_euclid_i64(-1_000_003), 0);

let n = BigUint::from_u128(1u128 << 127);
assert_eq!(n.rem_reciprocal(&r), n.rem_u64(1_000_003));
```

3 Signed integers: `BigInt` and `Sign`

A `BigInt` is a `Sign` (`Negative`, `Zero`, `Positive`) joined to a `BigUint` magnitude, with zero normalized to `Sign::Zero`. Construct with `from_biguint` (non-negative), `from_parts`, `from_i64`, or `from_i128` (total over its range: `i128::MIN`'s magnitude 2^{127} is an ordinary `u128`); read back

with `sign()` and `magnitude()`; `negated` flips the sign. `add` / `sub` are signed with in-place forms; `mul` is the full signed product and `mul_biguint` scales by an unsigned factor; `abs` returns the magnitude as a `BigUint`. `Ord` is sign-aware: negatives below zero below positives, magnitudes compared within a sign.

`div_rem` divides with remainder, *truncated toward zero* — the convention of C and of Rust's primitive `/` and `%`:

$$a \;=\; q\,d + r, \qquad |r| < |d|, \qquad \operatorname{sign}(r) \in \{\operatorname{sign}(a), 0\}, \tag{10}$$

so (-7).`div_rem`$(2) = (-3, -1)$ where floored division would give $(-4, 1)$. It panics on a zero divisor. `rem_positive` is the floored counterpart against an unsigned modulus, mapping into the canonical range $[0, n)$:

$$a \bmod {}^{+} n \;=\; a - n\left\lfloor \frac{a}{n} \right\rfloor \in [0, n), \tag{11}$$

which extended Euclid and every modular routine here need (a negative representative would poison a Montgomery encoding).

```
let ten = BigInt::from_biguint(BigUint::from_u64(10));
let minus_three = BigInt::from_parts(Sign::Negative, BigUint::from_u64(3));
assert_eq!(ten.add(&minus_three), BigInt::from_biguint(BigUint::from_u64(7)));

// The signed ring: full product, truncated division, absolute value.
assert_eq!(
    minus_three.mul(&ten),
    BigInt::from_parts(Sign::Negative, BigUint::from_u64(30))
);
let minus_seven = BigInt::from_parts(Sign::Negative, BigUint::from_u64(7));
let two = BigInt::from_biguint(BigUint::from_u64(2));
let (q, r) = minus_seven.div_rem(&two);
assert_eq!(q, BigInt::from_parts(Sign::Negative, BigUint::from_u64(3)));
assert_eq!(r, BigInt::from_parts(Sign::Negative, BigUint::one()));
assert_eq!(minus_seven.abs(), BigUint::from_u64(7));

// -3 = 8 (mod 11), in canonical range.
assert_eq!(minus_three.rem_euclid(&BigUint::from_u64(11)), BigUint::from_u64(8));
```

4 Barrett reduction: `BarrettContext`

`BarrettContext` is the fixed-modulus reduction context for a modulus of *either parity* — the complement to `MontgomeryContext`, which requires an odd one. `BarrettContext::new(n)` returns `Err(ModulusError::Zero)` or `Err(ModulusError::One)` for $n < 2$; otherwise it pays one division to precompute, for a k-limb modulus,

$$\mu \;=\; \left\lfloor \frac{\beta^{2k}}{n} \right\rfloor. \tag{12}$$

`reduce(x)`, for $0 \le x < \beta^{2k}$, estimates the quotient with two multiplications and no division

(Handbook of Applied Cryptography, Algorithm 14.42):

$$\hat{q} \;=\; \left\lfloor \frac{\left\lfloor x/\beta^{\,k-1} \right\rfloor \cdot \mu}{\beta^{\,k+1}} \right\rfloor, \qquad q-2 \;\le\; \hat{q} \;\le\; q \;=\; \left\lfloor \frac{x}{n} \right\rfloor, \tag{13}$$

so $x - \hat{q}\,n$ lies in $[0, 3n)$ and at most two conditional subtractions finish the reduction. An $x \ge \beta^{2k}$ falls back to the division the context exists to avoid.

`mod_mul`, `mod_square`, and `mod_pow` are built on `reduce`. `mod_pow` is left-to-right binary exponentiation (Knuth, §4.6.3) with the accumulator seeded from the exponent's top set bit — one squaring per remaining exponent bit and one multiplication per remaining set bit, each followed by a reduction. $0^0 = 1$ by the usual convention.

```
let even = BigUint::from_u64(1_000);
let ctx = BarrettContext::new(&even).expect("modulus is at least 2");
assert_eq!(
    ctx.mod_mul(&BigUint::from_u64(123), &BigUint::from_u64(456)),
    BigUint::from_u64(88) // 56 088 mod 1000
);
```

5 The Montgomery domain: `MontgomeryContext`

Montgomery multiplication (Montgomery 1985) replaces division by a modulus n with division by a power of the limb base, which is a shift. Fix an odd modulus n of k limbs and let $R = \beta^{\,k}$, so $\gcd(R, n) = 1$. The *Montgomery representative* of x is

$$\bar{x} \;=\; xR \bmod n. \tag{14}$$

`MontgomeryContext::new(n)` refuses a zero modulus with `Err(ModulusError::Zero)` and an even one with `Err(ModulusError::Even)`; otherwise it precomputes $R^2 \bmod n$ (for encoding) and the word-level constant (Dussé and Kaliski, EUROCRYPT '90)

$$n'_0 \;=\; -\,n_0^{-1} \bmod \beta, \tag{15}$$

where n_0 is the low limb of n.

REDC. The core operation: given $0 \le T < nR$,

$$\begin{aligned} m \;&=\; \left((T \bmod R)\,n'\right) \bmod R, \qquad && n' \equiv -n^{-1} \pmod R, \\ t \;&=\; \frac{T + m\,n}{R}, && t \;\equiv\; TR^{-1} \pmod n,\; 0 \le t < 2n, \end{aligned} \tag{16}$$

followed by one conditional subtraction. The division by R is exact by the choice of m ($T + mn \equiv 0 \bmod R$), and word-level interleaving needs only n'_0, never the full n'. Products in the domain multiply as

$$\mathrm{mul_mont}(\bar{x}, \bar{y}) \;=\; \mathrm{REDC}(\bar{x}\,\bar{y}) \;=\; \overline{xy}, \tag{17}$$

so a long computation encodes once ($\bar{x} = \mathrm{REDC}(x \cdot R^2)$), stays in the domain, and decodes once at the boundary ($x = \mathrm{REDC}(\bar{x})$).

A domain value is an opaque `MontgomeryResidue`, produced only by `to_residue` and read only by `from_residue`, with `one()` giving $\bar{1} = R \bmod n$. The domain operations are `mul_residue`, `square_residue`, `add_residue`, `sub_residue` and `pow_residue`; the encoding is linear, so domain addition and subtraction are one compare-and-correct each.

Belonging is by provenance: a context and its clones share an identity, while a context rebuilt from the same modulus is a different one and refuses residues it did not make.

The opacity is the point. A domain value must be encoded, reduced, and bound to one context, and none of those can be stated in a signature that takes a bare integer. The type carries all three instead. Where the relationship cannot be checked at compile time, two contexts both built at run time, it is checked at run time and the operations return `Result<_, ContextMismatch>`.

`mul`, `square`, and `pow` are one-shot forms that convert internally. For loops where the product *is* the loop, every domain operation has a `_with` form taking an opaque `MontgomeryScratch`, threading one caller-owned buffer through a sequence instead of allocating per multiply. The exponentiation ladder has two engines, chosen by exponent width: above 64 bits, left-to-right k-ary with a fixed window of `WINDOW` $= 4$ bits (`src/bigint/montgomery.rs`) over a sixteen-entry power table held in the domain (HAC Algorithm 14.82), seeded from the top window; at 64 bits or fewer (e.g. the F4 public exponent 65 537), right-to-left binary square-and-multiply, which skips the window-table setup that would dominate so short a ladder.

Contract 5.1 (The domain invariant lives in the type)**.** *A `MontgomeryResidue` is encoded, reduced below the modulus, and bound to the context that produced it. There is no constructor that can violate this and no accessor that exposes the raw value, so no domain operation has to re-check it. A residue offered to a context that did not produce it is refused with `ContextMismatch` rather than silently producing a wrong answer. The boundary operations `to_residue` / `from_residue` accept any representative and reduce.*

```
let p = BigUint::from_u64(97);
let ctx = MontgomeryContext::new(&p).expect("97 is odd");
let a = BigUint::from_u64(5);
let b = BigUint::from_u64(6);

// One-shot operations convert in and out internally.
assert_eq!(ctx.mul(&a, &b), BigUint::from_u64(30));
assert_eq!(ctx.pow(&a, &BigUint::from_u64(3)), BigUint::from_u64(28)); // 125 mod 97

// Staying in the domain: encode once, operate cheaply, decode once.
let a_mont = ctx.to_residue(&a);
let b_mont = ctx.to_residue(&b);
let product = ctx.mul_residue(&a_mont, &b_mont).expect("same context");
assert_eq!(ctx.from_residue(&product).expect("same context"), BigUint::from_u64(30));


// Loops thread one scratch buffer through the domain operations.
let mut scratch = MontgomeryScratch::new();
assert_eq!(
    ctx.mul_residue_with(&a_mont, &b_mont, &mut scratch).expect("same context"),
    product
);

let difference = ctx.sub_residue(&a_mont, &b_mont).expect("same context");
```

```
assert_eq!(ctx.from_residue(&difference).expect("same context"), BigUint::from_u64
    (96));

// A residue from another context is refused, not silently wrong.
let other = MontgomeryContext::new(&BigUint::from_u64(101)).expect("101 is odd");
assert!(other.from_residue(&a_mont).is_err());
```

6 Binary fields: `Gf2m`

A `Gf2m` is the field $\mathbb{F}_{2^m} = \mathbb{F}_2[x]/(f)$ for an irreducible $f \in \mathbb{F}_2[x]$ of degree m. Field elements and the defining polynomial are `BigUint` bit patterns: bit i is the coefficient of x^i. `Gf2m::new(f)` derives $m = \deg f$ and returns `None` for constants; *irreducibility is the caller's contract* — `Gf2m::is_irreducible` (below) guards it for untrusted polynomials. On a reducible modulus the ring $\mathbb{F}_2[x]/(f)$ is not a field: `inverse` and `div` report non-units as `None`, `solve_quadratic` verifies its root, and `trace` *panics* in every build when the Frobenius sum leaves $\{0, 1\}$ (a sum that is possible only outside a field); `sqrt` alone silently returns a wrong value there, because squaring is a bijection only in a field and its signature can carry neither a panic nor a `None`.

6.1 Arithmetic

Addition is coefficient-wise XOR — `Gf2m::add` is an associated function, since it needs no modulus. Multiplication is the word-level comb with reduction (Guide to Elliptic Curve Cryptography, Algorithm 2.36); reduction folds one excess word at a time through the *tap identity*: writing $f = x^m + \sum_{t \in T} x^t$ with T the lower exponents,

$$x^m \equiv \sum_{t \in T} x^t \pmod{f}, \tag{18}$$

which is $\mathbb{F}_2$-linear and therefore applies to 64 coefficients at once — one shifted XOR per tap per word, four or fewer for the trinomial and pentanomial moduli every standard uses. Squaring is linear in characteristic 2,

$$\Big(\sum c_i x^i\Big)^2 = \sum c_i\, x^{2i}, \tag{19}$$

implemented by interleaving a zero bit after every coefficient bit through a spread table, then reducing. `inverse` is the extended Euclidean algorithm over $\mathbb{F}_2[x]$ (Guide to ECC, Algorithm 2.48), maintaining $u = b \cdot a + s \cdot f$ with only the cofactor of a tracked; `div(a, b)` is $a \cdot b^{-1}$, `None` for a non-unit divisor. `pow` is binary exponentiation over the field.

```
// GF(2^3) with x^3 + x + 1.
let field = Gf2m::new(BigUint::from_u64(0b1011)).expect("degree 3");
assert_eq!(field.degree(), 3);

// (x + 1)(x^2 + 1) = x^3 + x^2 + x + 1 = x^2 (mod f).
let x_plus_1 = BigUint::from_u64(0b011);
let x2_plus_1 = BigUint::from_u64(0b101);
assert_eq!(field.mul(&x_plus_1, &x2_plus_1), BigUint::from_u64(0b100));

// x * (x^2 + 1) = x^3 + x = 1, so they are inverses.
let x = BigUint::from_u64(0b010);
```

```
assert_eq!(field.inverse(&x), Some(x2_plus_1));
assert_eq!(field.inverse(&BigUint::zero()), None);
```

6.2 Trace, half-trace, square roots, and quadratics

The *absolute trace* is the $\mathbb{F}_2$-linear map

$$\mathrm{Tr}(c) \;=\; \sum_{i=0}^{m-1} c^{2^i} \;\in\; \mathbb{F}_2, \tag{20}$$

computed by $m-1$ squarings and XORs (`trace`). Because squaring is a field automorphism (Frobenius), every element has a unique square root,

$$\sqrt{c} \;=\; c^{2^{m-1}}, \tag{21}$$

computed by $m-1$ squarings (`sqrt`).

The quadratic $z^2 + z = c$ is solvable exactly when $\mathrm{Tr}(c) = 0$ (the map $z \mapsto z^2 + z$ is 2-to-1 onto the trace-zero hyperplane), and its two roots differ by 1. For odd m the *half-trace*

$$\mathrm{HT}(c) \;=\; \sum_{i=0}^{(m-1)/2} c^{2^{2i}} \qquad \text{satisfies} \qquad \mathrm{HT}(c)^2 + \mathrm{HT}(c) \;=\; c + \mathrm{Tr}(c), \tag{22}$$

so `half_trace` solves the quadratic directly when $\mathrm{Tr}(c) = 0$; it panics at even degree, where the identity collapses (every FIPS binary curve degree — 163, 233, 283, 409, 571 — is odd). `solve_quadratic` is the total form: it tests the trace, works at every degree (even degree via IEEE Std 1363-2000, Annex A.4.7), and verifies its root. Over an irreducible modulus it returns `None` exactly when $\mathrm{Tr}(c) = 1$; over a reducible one, also whenever the trace computation, the auxiliary trace-one element, or the final verification fails — the root-verifying design is what keeps a non-field from producing a silently wrong answer here.

```
// Tr(x) = 0 in GF(2^3), so the half-trace solves z^2 + z = x.
let z = field.half_trace(&x);
assert_eq!(Gf2m::add(&field.square(&z), &z), x);

// Squaring is a bijection; sqrt inverts it.
assert_eq!(field.sqrt(&BigUint::from_u64(0b100)), x);

// solve_quadratic is total across degrees - here in the AES byte field
// GF(2^8), where the degree is even and the half-trace does not apply.
let aes = Gf2m::new(BigUint::from_u64(0x11B)).expect("the AES byte field");
let a = BigUint::from_u64(0x53);
let c = Gf2m::add(&aes.square(&a), &a);
let z = aes.solve_quadratic(&c).expect("solvable by construction");
assert_eq!(Gf2m::add(&aes.square(&z), &z), c);
```

6.3 Irreducibility: Rabin's test

`Gf2m::is_irreducible(f)` decides irreducibility deterministically (Rabin 1980, the $\mathbb{F}_2$ form). The criterion rests on the identity that $x^{2^k} - x$ is the product of all monic irreducibles over $\mathbb{F}_2$ of

degree dividing k: a polynomial f of degree m is irreducible if and only if

$$x^{2^m} \equiv x \pmod{f} \qquad \text{and} \qquad \gcd(x^{2^{m/q}} - x,\ f) = 1 \ \text{ for every prime } q \mid m. \tag{23}$$

The first condition confines every irreducible factor's degree to a divisor of m; the gcd conditions exclude the proper divisors (every proper divisor of m divides some m/q). Mechanically $x^{2^i} \bmod f$ is i applications of `square` to x, so one forward pass of m squarings visits the checkpoints m/q in ascending order and finishes at x^{2^m} — m squarings plus one polynomial gcd per distinct prime divisor of m.

```
assert!(Gf2m::is_irreducible(&BigUint::from_u64(0b1011)));  // x^3 + x + 1
assert!(!Gf2m::is_irreducible(&BigUint::from_u64(0b1001)); // x^3 + 1
assert!(Gf2m::is_irreducible(&BigUint::from_u64(0x11B)));   // AES
```

7 Number theory

7.1 Greatest common divisors

`gcd(a, b)` computes $\gcd(a, b)$ with $\gcd(a, 0) = a$; `lcm(a, b)` $= ab/\gcd(a, b)$, with $\operatorname{lcm}(a, 0) = 0$. `gcd_u64` is the word-sized form — single-word Euclid, the base case the wide `gcd` falls to, public so callers holding machine words skip the heap entirely. `gcd_extended` returns the Bézout triple (g, s, t) of signed cofactors with

$$g \ = \ \gcd(a, b) \ = \ as + bt. \tag{24}$$

Three engines back these, chosen by size. Small operands run Euclid with word-level steps. Above that, Lehmer's algorithm (Knuth, §4.5.2, Algorithm L) simulates a whole run of Euclidean quotients on the leading words: each quotient extends a 2×2 integer transform

$$M \ = \ \begin{pmatrix} q & 1 \\ 1 & 0 \end{pmatrix}_k \cdots \begin{pmatrix} q & 1 \\ 1 & 0 \end{pmatrix}_1 , \tag{25}$$

and the batch is certified against both endpoints of the leading-word interval before being replayed once, at full width, as a single 2×2 matrix–vector application (four scalar-by-bignum products). The window is `LEADING_DIGIT_BITS` $= 124$ bits wide, as wide as the `i128` transform arithmetic allows, and Lamé's bound caps a batch at `LEHMER_BATCH_MAX_STEPS` $= 178$ quotients. From `HGCD_THRESHOLD_LIMBS` (2048 limbs in the smaller operand) `gcd` switches to the subquadratic Half-GCD (Möller 2008): recursively compute the transform that reduces the top halves, splice it onto the full operands, and recurse, for $T(n) = O(M(n) \log n)$ where M is the multiplication cost; the recursion bottoms out in Lehmer batches below `HGCD_BASE_LIMBS` (96 limbs). `gcd_extended` crosses over lower, at `HGCD_EXT_THRESHOLD_LIMBS` (512 limbs), because Lehmer's extended form carries full-width cofactors through every batch; above it the driver's Bézout pair is canonicalized so the triple returned is the classical one, s the representative of least absolute value modulo b/g.

```
let a = BigUint::from_u64(240);
let b = BigUint::from_u64(46);
assert_eq!(gcd(&a, &b), BigUint::from_u64(2));
assert_eq!(
    lcm(&BigUint::from_u64(4), &BigUint::from_u64(6)),
```

```
    BigUint::from_u64(12)
);

let (g, s, t) = gcd_extended(&a, &b);
let bezout = s.mul_biguint(&a).add(&t.mul_biguint(&b));
assert_eq!(bezout, BigInt::from_biguint(g));

// The word-sized form answers without an allocation.
assert_eq!(gcd_u64(240, 46), 2);
assert_eq!(gcd_u64(0, 7), 7); // gcd(0, b) = b
```

7.2 Quadratic-residue symbols

For an odd prime p the Legendre symbol is

$$\left(\frac{a}{p}\right) = \begin{cases} 0 & p \mid a, \\ +1 & a \text{ is a non-zero square mod } p, \\ -1 & \text{otherwise,} \end{cases} \qquad \left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod{p}, \tag{26}$$

the congruence being Euler's criterion. The Jacobi symbol extends it multiplicatively to odd $n = \prod p_i^{e_i}$ as $\left(\frac{a}{n}\right) = \prod \left(\frac{a}{p_i}\right)^{e_i}$; `jacobi(a, n)` computes it without factoring n, from the Euclidean quotient sequence of (a mod n, n). Quadratic reciprocity and the supplement for 2,

$$\left(\frac{a}{n}\right)\left(\frac{n}{a}\right) = (-1)^{\frac{a-1}{2}\cdot\frac{n-1}{2}}, \qquad \left(\frac{2}{n}\right) = (-1)^{\frac{n^2-1}{8}}, \tag{27}$$

together with periodicity in the top argument, make each Euclidean step $x = qy + r$ a change of sign that depends only on q and on x, y mod 4, so a small state machine replaying the applied quotients carries the symbol while the remainders shrink (Schönhage's identities, in the form of Möller's design as realized in GMP's `mpn_hgcd_jacobi`). The quotient sequence runs on Lehmer batches, and on machine words once both remainders fit one; from `JACOBI_HGCD_THRESHOLD_LIMBS` (2048 limbs) the state threads through the Half-GCD recursion and the symbol costs $O(M(n) \log n)$, as `gcd` does. It returns `None` for even n, where the symbol is undefined; `legendre` is the same computation under its prime-modulus name; `kronecker` extends to every modulus through $\left(\frac{a}{2}\right) = 0$ for even a and $(-1)^{(a^2-1)/8}$ for odd, with $\left(\frac{a}{0}\right) = 1$ exactly when $a = 1$.

Note $\left(\frac{a}{n}\right) = +1$ does not certify that a is a residue for composite n — the two factors' symbols can both be -1. Only $\left(\frac{a}{n}\right) = -1$ certifies non-residuosity.

```
// (2/9) = 1 because 9 = 1 (mod 8); (3/9) = 0 by the shared factor.
assert_eq!(jacobi(&BigUint::from_u64(2), &BigUint::from_u64(9)), Some(1));
assert_eq!(jacobi(&BigUint::from_u64(3), &BigUint::from_u64(9)), Some(0));
// 2 is a residue mod 7 (3^2 = 9 = 2).
assert_eq!(legendre(&BigUint::from_u64(2), &BigUint::from_u64(7)), Some(1));
// Kronecker handles even moduli: (5/8) = (5/2)^3 = -1.
assert_eq!(kronecker(&BigUint::from_u64(5), &BigUint::from_u64(8)), -1);
```

7.3 Modular exponentiation and inverses

`mod_pow(b, e, n)` computes b^e mod n for any $n \geq 1$: through the Montgomery ladder when n is odd, and over a `BarrettContext` when n is even (where Montgomery cannot operate). Neither

route divides per step. A caller holding one even modulus across many exponentiations should build the `BarrettContext` itself and keep it, since the context costs one division to precompute μ, which `mod_pow` pays per call.

`mod_inverse(a, n)` returns $a^{-1} \bmod n$ from the extended Euclidean identity: when $\gcd(a, n) = 1$, the cofactor s in $as + nt = 1$ reduces to the inverse; when the gcd exceeds one there is no inverse and the result is `None`.

`mod_inverse_u64` is the word-sized companion: extended Euclid with the Bézout coefficients carried in `i128` — the intermediate $s_{k-1} - q_k s_k$ is not bounded by `u64` and would wrap there — so it is total over `u64` and panics only on a zero modulus. `mod_inverse_u128` is the double-word sibling, total over every `u128` modulus: signed cofactors would need a 129th bit past 2^{127}, so it carries their magnitudes unsigned — the signs of the coefficient sequence alternate, so each step adds magnitudes and the parity of the step count gives the final sign — and hands a modulus that fits a word to `mod_inverse_u64`.

```
assert_eq!(mod_inverse_u64(3, 7), Some(5));
assert_eq!(mod_inverse_u64(2, 4), None); // shares a factor

// The double-word companion covers every u128 modulus, past 2^127 too.
let m = (1u128 << 127) + 45; // m ≡ 2 (mod 3), so 3 · (m + 1)/3 ≡ 1
assert_eq!(mod_inverse_u128(3, m), Some((m + 1) / 3));
```

`mod_inverse_batch` inverts a whole slice at the cost of one inversion and $3(k-1)$ multiplications — Montgomery's trick. With prefix products $P_i = x_1 x_2 \cdots x_i$,

$$P_k^{-1} \text{ (one inversion)}, \qquad x_i^{-1} = P_{i-1} \cdot P_i^{-1}, \qquad P_{i-1}^{-1} = x_i \cdot P_i^{-1}, \tag{28}$$

walking the products back from the top. `None` when any element shares a factor with the modulus (which element is not identified — that would cost the inversions the trick avoids).

```
assert_eq!(
    mod_pow(&BigUint::from_u64(3), &BigUint::from_u64(4), &BigUint::from_u64(5)),
    BigUint::one() // 81 = 1 (mod 5)
);
assert_eq!(
    mod_inverse(&BigUint::from_u64(3), &BigUint::from_u64(7)),
    Some(BigUint::from_u64(5)) // 3 * 5 = 15 = 1 (mod 7)
);
assert_eq!(mod_inverse(&BigUint::from_u64(2), &BigUint::from_u64(4)), None);

let m = BigUint::from_u64(97);
let values = [BigUint::from_u64(3), BigUint::from_u64(10), BigUint::from_u64(96)];
let inverses = mod_inverse_batch(&values, &m).expect("all coprime to 97");
for (inv, v) in inverses.iter().zip(&values) {
    assert!(BigUint::mod_mul(inv, v, &m).is_one());
}
```

7.4 Square roots rem a prime

`mod_sqrt(a, p)` returns a square root of a rem an odd prime p, or `None` for a non-residue. Write $p - 1 = 2^s q$ with q odd. Three engines serve the call, chosen by the 2-adic depth s.

The $p \equiv 3 \pmod 4$ shortcut. When $s = 1$ — half of all primes, so the branch that runs most often — the root is a single exponentiation: for a residue a,

$$r \;=\; a^{(p+1)/4} \pmod p, \qquad r^2 \;=\; a^{(p+1)/2} \;=\; a \cdot a^{(p-1)/2} \;=\; a, \tag{29}$$

the last step by Euler's criterion.

Tonelli–Shanks. For deeper s, initialize with a non-residue z (found by a deterministic scan testing $\left(\frac{z}{p}\right) = -1$; $(p-1)/2$ of the non-zero residues qualify, and the scan is abandoned after `NON_RESIDUE_SCAN_BOUND` $= 128$ misses, which no prime modulus reaches with probability above 2^{-128}):

$$M = s, \quad c = z^q, \quad t = a^q, \quad r = a^{(q+1)/2} \pmod p. \tag{30}$$

The invariants $r^2 = a\,t$ and $\operatorname{ord}(t) \mid 2^{M-1}$ hold throughout; while $t \neq 1$, find the least i with $t^{2^i} = 1$, set $b = c^{2^{M-i-1}}$, and update

$$M \leftarrow i, \qquad c \leftarrow b^2, \qquad t \leftarrow t b^2, \qquad r \leftarrow r b. \tag{31}$$

Each pass strictly reduces M, so at most s passes run; when $t = 1$, $r^2 = a$.

Cipolla. When the 2-adic depth is large — $s^2 > $ `CIPOLLA_THRESHOLD_FACTOR` $\cdot \lceil \lg p \rceil$, with the factor 4 — the descent's $O(s^2)$ multiplications dominate, and the implementation dispatches to Cipolla's algorithm: find t with $\left(\frac{t^2-a}{p}\right) = -1$, work in $\mathbb{F}_{p^2} = \mathbb{F}_p(\omega)$ with $\omega^2 = t^2 - a$, and compute

$$r \;=\; (t+\omega)^{(p+1)/2}, \tag{32}$$

which lands in $\mathbb{F}_p$ and squares to a. Every path verifies its result by squaring before returning it, so on a composite modulus — for which the internal reasoning is unsound — the answer is either `None` or a genuine square root rem that composite, never a silently wrong value. `None` is *not* a compositeness certificate, and `Some` is not a primality one: `mod_sqrt`$(1, 15)$ returns `Some(1)`.

```
let p = BigUint::from_u64(41);
let root = mod_sqrt(&BigUint::from_u64(2), &p).expect("2 is a residue mod 41");
assert_eq!(BigUint::mod_mul(&root, &root, &p), BigUint::from_u64(2));
assert_eq!(mod_sqrt(&BigUint::from_u64(3), &p), None); // non-residue
```

7.5 Square roots rem a prime power

`mod_sqrt_prime_power(a, p, e)` returns *every* square root of a rem p^e, ascending, empty for a non-residue. For odd p and $p \nmid a$, a root mod p lifts uniquely along Hensel's construction: given $r_k^2 \equiv a \pmod{p^k}$,

$$r_{k+1} \;=\; r_k + t\,p^k, \qquad t \;=\; \frac{a - r_k^2}{p^k} \cdot (2r_k)^{-1} \bmod p, \tag{33}$$

and the two roots mod p give the two mod p^e. For $p = 2$ the structure is governed by the residue of a rem 8 (one root mod 2, two mod 4, and four mod 2^e for $e \geq 3$ when $a \equiv 1 \bmod 8$). For $p \mid a$, factor out the largest even power p^{2j} dividing a and recurse on the cofactor, multiplying the roots back by p^j and expanding the residual multiplicity. Panics when $e = 0$ or $p < 2$.

```
// 9 has four square roots mod 16: 3, 5, 11, 13.
let roots = mod_sqrt_prime_power(&BigUint::from_u64(9), &BigUint::from_u64(2), 4);
assert_eq!(roots, vec![
    BigUint::from_u64(3), BigUint::from_u64(5),
    BigUint::from_u64(11), BigUint::from_u64(13),
]);
```

7.6 The Chinese Remainder Theorem

`crt_combine` solves the simultaneous congruences $x \equiv a_i \pmod{m_i}$ for pairwise coprime moduli: the unique $x \bmod M$, $M = \prod m_i$, assembled by folding pairs,

$$x_{12} \;=\; a_1 + m_1\Big((a_2 - a_1)\, m_1^{-1} \bmod m_2\Big) \;\equiv\; a_i \pmod{m_i},\; i = 1, 2. \tag{34}$$

`None` when the moduli are not pairwise coprime (detected by the failing inversion).

`crt_combine_balanced` applies the same validation and returns the same canonical answer, but combines equal-width partial products up a balanced tree, running independent pairs on at most the requested number of workers — never more than the caller's budget, and serially when the request is zero. It is a `CrtBasis` made and used once: a caller with several vectors of residues over the same moduli makes the basis once and calls `combine` for each. The budget is the threads the calling thread's work may use, which `parallelism::budget` reports: the machine's count until `parallelism::with_budget` sets it for the work it is given, and a share of the caller's inside each of the crate's own fan-outs.

How many processors a machine reports does not say what its threads can do together: they share caches, and a block one thread reads from cheaply is read from dearly when every thread has one. `parallelism::gather_costs` measures it: what a word read from a place chosen at random costs, by the size of the block it is read from, with a given number of threads reading at once, each from a block of its own. The blocks double from 16 KB until the reads cost four times the cheapest; each is timed against a block of 16 KB read by the same thread in the same moments, so that a machine whose cores are not alike is not mistaken for one whose caches have ended. A measurement takes a fifth of a second or so and up to a gigabyte while it runs, and nothing is kept of it. `GatherCosts::block_within` names the largest block whose reads, and those of every smaller block, cost no more than a given multiple of the cheapest: the size to cut work into, for a routine that reads at random from something large.

`crt_combine_u64` is one Garner step at machine width: two congruences with word moduli, combined into the residue below their product, which always fits a `u128`; it returns `None` for a zero or non-coprime modulus.

```
// Sunzi's classic: 2 mod 3, 3 mod 5, 2 mod 7.
let x = crt_combine(&[
    (BigUint::from_u64(2), BigUint::from_u64(3)),
    (BigUint::from_u64(3), BigUint::from_u64(5)),
    (BigUint::from_u64(2), BigUint::from_u64(7)),
])
.expect("moduli are pairwise coprime");
assert_eq!(x, BigUint::from_u64(23));
assert_eq!(
    crt_combine_balanced(
        &[
```

```
            (BigUint::from_u64(2), BigUint::from_u64(3)),
            (BigUint::from_u64(3), BigUint::from_u64(5)),
            (BigUint::from_u64(2), BigUint::from_u64(7)),
        ],
        4,
    ),
    Some(x)
);

// Two word-sized congruences, no heap: 8 ≡ 2 (mod 3) and 8 ≡ 3 (mod 5).
assert_eq!(crt_combine_u64((2, 3), (3, 5)), Some(8));
assert_eq!(crt_combine_u64((1, 4), (3, 6)), None); // gcd(4, 6) = 2

// The moduli prepared once, for as many vectors of residues as there are.
let basis = CrtBasis::new(&[
    BigUint::from_u64(3),
    BigUint::from_u64(5),
    BigUint::from_u64(7),
])
.expect("moduli are pairwise coprime");
assert_eq!(*basis.modulus(), BigUint::from_u64(105));
let residues = |value: u64| [3u64, 5, 7].map(|m| BigUint::from_u64(value % m));
assert_eq!(basis.combine(&residues(23)), BigUint::from_u64(23));
assert_eq!(basis.combine(&residues(104)), BigUint::from_u64(104));

// A thread's budget is the machine's until it is set, and is put back.
let whole = budget();
assert_eq!(with_budget(1, budget), 1);
assert_eq!(budget(), whole);

// What a read at random costs here, two threads reading at once: blocks
// doubling from 16 KB, and the largest whose reads are within twice the
// cheapest.
let costs = gather_costs(2);
assert_eq!(costs.costs()[0].0, 16 << 10);
assert!(costs.block_within(2.0) >= costs.block_within(1.5));
```

7.7 Valuation

`valuation(n, p)` is the p-adic valuation $v_p(n)$, the exponent of p in n; `remove_factor(n, p)` returns $(n/p^{v_p(n)}, v_p(n))$, dividing through a ladder of squared powers $p, p^2, p^4, \ldots$ so a valuation of v costs $O(\lg v)$ divisions (the shape of GMP's `mpz_remove`). Both panic on $n = 0$ (the valuation is unbounded) or $p < 2$.

```
let n = BigUint::from_u64(3_888); // 2^4 * 3^5
assert_eq!(valuation(&n, &BigUint::from_u64(2)), 4);
let (cofactor, exponent) = remove_factor(&n, &BigUint::from_u64(3));
assert_eq!(exponent, 5);
assert_eq!(cofactor, BigUint::from_u64(16));
```

7.8 Rational reconstruction

Modular and p-adic computation ends by reading a rational answer back from its residue. Given x and m, `rational_reconstruct_bounded` finds the unique fraction p/q with

$$p \equiv q\,x \pmod{m}, \qquad |p| \le N, \quad 0 < q \le D, \quad \gcd(p,q) = 1, \tag{35}$$

under the caller's contract $2ND < m$, which is precisely what makes the answer unique (the difference of two candidate fractions would be a non-zero multiple of m over a denominator smaller than m). The contract is enforced: a call with $2ND \ge m$ *panics*, since any answer it produced could be one of several. The candidate comes from the extended Euclidean remainder sequence of (m, x), stopped at the first remainder $r \le N$: writing $r \equiv t\,x \pmod{m}$ for that row's cofactor t, the only possible answer is $p = \pm r$, $q = |t|$ (Wang 1981; von zur Gathen and Gerhard, §5.10). The final checks $q \le D$ and $\gcd(r, t) = 1$ decide existence.

`rational_reconstruct` is the symmetric special case $N = D = \lfloor\sqrt{(m-1)/2}\rfloor$. When reconstructing many values under one modulus, compute that bound once and call the bounded form — the wrapper's square root costs more than the reconstruction itself at large sizes.

```
// 22/7 survives reduction mod 1009 and comes back intact.
let m = BigUint::from_u64(1_009);
let seven_inv = mod_inverse(&BigUint::from_u64(7), &m).expect("7 is invertible");
let x = BigUint::mod_mul(&BigUint::from_u64(22), &seven_inv, &m);
let (p, q) = rational_reconstruct(&x, &m).expect("22/7 is within the bounds");
assert_eq!(p, BigInt::from_biguint(BigUint::from_u64(22)));
assert_eq!(q, BigUint::from_u64(7));

// The bounded form makes the contract explicit (here N = 22, D = 7).
assert_eq!(
    rational_reconstruct_bounded(&x, &m, &BigUint::from_u64(22), &BigUint::from_u64
    (7)),
    Some((BigInt::from_biguint(BigUint::from_u64(22)), BigUint::from_u64(7)))
);
```

7.9 Product trees, remainder trees, and batch smoothness

`product_tree(values)` builds the pairwise-product tree over its leaves — level 0 the values, each parent the product of its two children, the root $\prod_i x_i$ — and returns a `ProductTree`, whose levels are private: the descent below is correct only for the exact shape this constructor produces, so construction is the only way to obtain one. Read it back with `root()`, `leaves()`, `len()`, `is_empty()`, or `levels()`. `remainder_tree(tree, z)` reduces z down the tree, $z$mod each node, reaching $z \bmod x_i$ at every leaf for one tree descent instead of k full-width divisions; it panics on a zero leaf, which it would divide by.

`smooth_parts(values, primes)` is Bernstein's batched smoothness test. With $z = \prod_{p \in P} p$ (itself built by a product tree) and $r_i = z \bmod x_i$ from the remainder tree, square repeatedly,

$$s_i \;=\; r_i^{2^{e_i}} \bmod x_i, \qquad e_i = \lfloor \lg b_i \rfloor + 1 \text{ squarings for } b_i = \lfloor \lg x_i \rfloor + 1, \text{ so } 2^{e_i} > b_i \ge \lg x_i, \tag{36}$$

which pushes every smooth prime's multiplicity in s_i past its multiplicity in x_i (no prime can divide x_i more than $\lg x_i$ times). Then

$$\gcd(x_i, s_i) \;=\; \text{the largest divisor of } x_i \text{ composed of primes in } P, \tag{37}$$

the *smooth part*, with full multiplicity. A value is fully smooth exactly when its smooth part equals itself; $0 \mapsto 0$ and $1 \mapsto 1$ pass through without joining the tree.

`SmoothnessBase::new(P)` is the same computation with $z = \prod P$ built once and retained. The free function rebuilds z per call; for a base of a few thousand primes that product runs to tens of thousands of bits, which is free once per run but, charged per batch, dictates the batch size a caller may use. The context removes that coupling: batches may be as small as the caller likes and the answers do not depend on the grouping. The obligation that every entry of P is at least two is checked once, at construction.

```
let primes = primes_below(10); // 2, 3, 5, 7
let values = [
    BigUint::from_u64(360),    // 2^3 * 3^2 * 5 - fully smooth
    BigUint::from_u64(2 * 11), // 11 is not in the base
];
let parts = smooth_parts(&values, &primes);
assert_eq!(parts[0], BigUint::from_u64(360));
assert_eq!(parts[1], BigUint::from_u64(2));

// The same answers with the product hoisted, in batches of any size.
let base = SmoothnessBase::new(&primes).expect("entries at least two");
assert_eq!(base.primes(), &[2, 3, 5, 7]);
assert_eq!(base.smooth_parts(&values), parts);
assert_eq!(base.smooth_parts(&values[..1])[0], parts[0]);

// The trees are public on their own: the root is the product, and one
// descent reduces a modulus against every leaf.
let leaves = [BigUint::from_u64(7), BigUint::from_u64(11), BigUint::from_u64(13)];
let tree = product_tree(&leaves);
assert_eq!(*tree.root().unwrap(), BigUint::from_u64(7 * 11 * 13)); // 1001
assert_eq!(tree.len(), 3);
let residues = remainder_tree(&tree, &BigUint::from_u64(100));
assert_eq!(residues, vec![
    BigUint::from_u64(100 % 7),
    BigUint::from_u64(100 % 11),
    BigUint::from_u64(100 % 13),
]);
```

`primes_below(bound)` sieves every prime below the bound into a `Vec<u64>` — the bulk companion to the probabilistic tests below.

7.10 Primality

Miller–Rabin. For odd $n > 2$ write $n - 1 = 2^s d$ with d odd. A base $a \not\equiv 0, \pm 1 \pmod{n}$ is a *witness* to compositeness unless

$$a^d \equiv 1 \pmod{n} \qquad \text{or} \qquad a^{2^r d} \equiv -1 \pmod{n} \text{ for some } 0 \le r < s. \tag{38}$$

Every prime satisfies one of the two for every base; for a composite, at least three quarters of the bases are witnesses, so k random rounds err with probability at most 4^{-k}. `miller_rabin_witness(n, a)` is the single-round primitive: `true` means a proves n composite.

`is_probable_prime` runs trial division by the 168 primes below 1000, then Miller–Rabin over the twelve fixed bases of `MR_BASES`, the primes 2 through 37 — a deterministic proof below

$\psi_{12} \approx 3.19 \times 10^{23}$ (exactly 318 665 857 834 031 151 167 461: Sorenson and Webster 2017; the often-quoted 3.317×10^{24} is ψ_{13} and requires a thirteenth base) and a strong probable-prime test above. The same twelve bases, run in a word-width Montgomery domain, make `is_prime_u64` a deterministic test of every `u64`. `miller_rabin_with_bases` takes an explicit base set after the same unconditional sieve; each base is reduced rem the candidate, trivial residues $\{0, 1, n-1\}$ are discarded, and a set with no effective round proves nothing and reports composite. Fixed bases are for candidates you generated yourself: an adversary can construct pseudoprimes against any fixed base set, so untrusted input needs candidate-derived witnesses added on top.

The strong Lucas test and Baillie–PSW. Selfridge's Method A picks the discriminant D as the first of $5, -7, 9, -11, 13, \ldots$ (the integers $\equiv 1 \bmod 4$ with $|D| \geq 5$, ordered by absolute value) with $\left(\frac{D}{n}\right) = -1$, then sets $P = 1$, $Q = (1 - D)/4$. The search itself can prove n composite and stop: a zero symbol whose gcd with n is proper exposes a factor, and a perfect square — for which no D has symbol -1 and the search would never end — is detected by one integer square root after the third failed candidate and likewise reported composite (Baillie and Wagstaff, §6). The Lucas sequences

$$U_0 = 0,\ U_1 = 1,\ V_0 = 2,\ V_1 = P, \qquad U_{k+1} = P\,U_k - Q\,U_{k-1}, \quad V_{k+1} = P\,V_k - Q\,V_{k-1}, \tag{39}$$

are computed rem n by the doubling and stepping identities

$$U_{2k} = U_k V_k, \quad V_{2k} = V_k^2 - 2Q^k, \quad U_{k+1} = \tfrac{PU_k + V_k}{2}, \quad V_{k+1} = \tfrac{DU_k + PV_k}{2}, \tag{40}$$

the halvings exact rem odd n. Writing $n + 1 = 2^s d$ with d odd, n is a *strong Lucas probable prime* when

$$U_d \equiv 0 \pmod{n} \qquad \text{or} \qquad V_{2^r d} \equiv 0 \pmod{n} \text{ for some } 0 \leq r < s, \tag{41}$$

conditions every prime with $\left(\frac{D}{n}\right) = -1$ satisfies (Baillie and Wagstaff 1980). This stage alone is exposed as `is_strong_lucas_probable_prime`.

`is_probable_prime_bpsw` is the Baillie–PSW composite of the two: trial division, one strong base-2 Miller–Rabin round, then the strong Lucas test. The two stages fail on disjoint kinds of composites as far as anyone has found: no composite passing both is known, and none exists below 2^{64}, where the test is therefore deterministic.

```
assert!(is_probable_prime(&BigUint::from_u64(65_537)));
assert!(!is_probable_prime(&BigUint::from_u64(561))); // Carmichael

// 2047 = 23 * 89 is the smallest strong pseudoprime to base 2.
let n = BigUint::from_u64(2_047);
assert!(!miller_rabin_witness(&n, &BigUint::from_u64(2))); // 2 is fooled
assert!(miller_rabin_witness(&n, &BigUint::from_u64(3)));  // 3 is a witness

// with_bases reduces each base rem n and discards {0, 1, n-1}; a set
// of only trivial bases runs no effective round and is not reported prime.
let composite = BigUint::from_u64(1_022_117); // 1009 x 1013, survives the sieve
assert!(!miller_rabin_with_bases(&composite, &[1])); // 1 never testifies
assert!(!miller_rabin_with_bases(&composite, &[2])); // 2 exposes it

// BPSW: the stages reject disjoint composites.
```

```
assert!(!is_probable_prime_bpsw(&BigUint::from_u64(2_047))); // Lucas rejects
assert!(is_strong_lucas_probable_prime(&BigUint::from_u64(5_459)));
assert!(!is_probable_prime_bpsw(&BigUint::from_u64(5_459))); // base 2 rejects
```

The general Lucas test of FIPS 186-4. Appendix C.3.3 of FIPS 186-4 specifies a Lucas test with a single acceptance condition, and `is_lucas_probable_prime` follows it step for step. A perfect square is composite (step 1; `is_square` answers Appendix C.4's question exactly). D is the first of $5, -7, 9, -11, \ldots$ with $\left(\frac{D}{n}\right) = -1$, and a zero symbol means composite (step 2). Write $K = n + 1 = \sum_{i=0}^{r} K_i 2^i$ with $K_r = 1$. From $U = V = 1$, for $i = r - 1$ down to 0, set

$$U' = UV, \qquad V' = \tfrac{V^2 + DU^2}{2}, \tag{42}$$

then $U \leftarrow \frac{U'+V'}{2}$ and $V \leftarrow \frac{V'+DU'}{2}$ when $K_i = 1$, and $U \leftarrow U'$, $V \leftarrow V'$ otherwise — all rem n, with an odd A halved as $(A + n)/2$ (steps 3–6). The candidate is accepted when the final U is zero (step 7), which is $U_{n+1} \equiv 0 \pmod{n}$ for the sequences above with $P = 1$, $Q = (1 - D)/4$. Since

$$U_{n+1} = U_d \, V_d \, V_{2d} \cdots V_{2^{s-1}d}, \tag{43}$$

every candidate the strong test accepts is accepted here, but not conversely: $323 = 17 \cdot 19$ is the first composite this test passes (OEIS A217120), and the strong test rejects it.

C.3.3 is specified for odd candidates; the function also answers 0, 1, and even inputs correctly (2 is prime, the rest are not). Step 2 is read with one qualification, which can only matter for a candidate no larger than the $|D|$ its own search reaches. A zero symbol means $\gcd(D, n) > 1$, which proves n composite when the gcd is a proper divisor — always the case while $|D| < n$. When n itself divides D the symbol proves nothing, and only a prime can reach such a D: a composite non-square n has a least prime factor $p < n$, and the search meets $|D| = p$ (or $|D| = 9$, when $p = 3$) first and stops there. Read literally, the step would report the primes 5 and 11 composite, at $D = 5$ and $D = -11$, so the function passes over such a symbol. For every candidate larger than the $|D|$ its search reaches — every candidate FIPS 186-4 tests — the result is the text's exactly.

```
assert!(is_lucas_probable_prime(&BigUint::from_u64(65_537)));
assert!(is_lucas_probable_prime(&BigUint::from_u64(11))); // passes over D = -11
assert!(!is_lucas_probable_prime(&BigUint::from_u64(10_201))); // 101^2

// 323 = 17 * 19 is a Lucas pseudoprime but not a strong one.
assert!(is_lucas_probable_prime(&BigUint::from_u64(323)));
assert!(!is_strong_lucas_probable_prime(&BigUint::from_u64(323)));
```

AKS. `is_prime_aks` is the exact deterministic test of Agrawal, Kayal, and Saxena (2004). It rejects perfect powers, finds a modulus r with $\mathrm{ord}_r(n) > \lg^2 n$, checks small gcds, and then verifies

$$(X + a)^n \equiv X^n + a \pmod{X^r - 1,\ n} \tag{44}$$

for enough consecutive values of a. A `true` result is an unconditional primality proof, not a probable-prime verdict, and `false` means composite. The constants are large: the probable-prime tests above are the practical ones, and AKS is for callers that need this algorithm specifically.

```rust
// Unlike the probable-prime predicates, AKS is an unconditional proof.
assert!(is_prime_aks(&BigUint::from_u64(101)));
assert!(!is_prime_aks(&BigUint::from_u64(561))); // Carmichael
```

8 Polynomials: `PolyZ` and `PolyMod`

Both types are dense univariate polynomials, coefficients stored low to high and normalized to drop trailing zeros, so the zero polynomial is the empty coefficient list and `degree()` is `None` for it. `PolyZ` works over $\mathbb{Z}$; `PolyMod` over $\mathbb{Z}/m\mathbb{Z}$ with every coefficient held as its least non-negative residue and the modulus carried by the value — mixing two moduli in one operation is a hard panic in every build. The division-based operations of `PolyMod` require a *prime* modulus (they invert leading coefficients); a composite modulus panics on the first non-invertible pivot.

8.1 Arithmetic over $\mathbb{Z}$

`add`/`sub`/`mul` (the coefficient convolution: schoolbook below `POLY_KARATSUBA_THRESHOLD_Z` (96 coefficients), Karatsuba above, with two further clauses. The crossover is measured over operand *ratios*, not balanced pairs alone, because a split at half the longer operand saves almost nothing when the shorter one barely reaches past it — so between 96 and `POLY_KARATSUBA_ANY_RATIO_Z` (128 coefficients) only pairs within the ratio 5 : 4 split, and from 128 every ratio does. And the split is refused to an operand too *sparse* to pay for it, since the schoolbook pass skips a zero coefficient's whole inner loop and the split's sums destroy that saving; the density a pair must reach is computed from the two kernels' own coefficient-product counts and so falls as the operands grow, from three quarters at 96 coefficients to under a quarter at 2048), `evaluate` (Horner: fold $acc \leftarrow acc \cdot x + c_i$ from the top — one multiplication and addition per coefficient), `derivative` (the formal rule $\sum i\, c_i x^{i-1}$), `negated`, and `scale`. `content` is the gcd of the coefficients; `primitive_part` divides it out, by Gauss's lemma the canonical split $f = \mathrm{cont}(f) \cdot \mathrm{pp}(f)$.

8.2 Division over $\mathbb{Z}$

$\mathbb{Z}$ is not a field, so two divisions exist.

Exact division. `div_rem` returns (q, r) with $f = q\,g + r$, $\deg r < \deg g$, when an integer quotient exists, and `None` otherwise: each long-division step must divide the running remainder's leading coefficient exactly by $\mathrm{lc}(g)$ (always possible for monic g). The quotient is unique when it exists, so a single failing step is conclusive.

Pseudo-division. `pseudo_div_rem` premultiplies instead of dividing (Knuth, Algorithm R), staying in $\mathbb{Z}$ unconditionally:

$$\ell\, f \;=\; q\, g + r, \qquad \deg r < \deg g, \qquad \ell = \mathrm{lc}(g)^{\deg f - \deg g + 1} \tag{45}$$

(and $\ell = 1$ when $\deg f < \deg g$, where the result is $(0, f)$).

```rust
// (x + 1)(x + 2) = x^2 + 3x + 2, over Z.
let a = PolyZ::from_i64_slice(&[1, 1]); // x + 1
```

```
let b = PolyZ::from_i64_slice(&[2, 1]); // x + 2
assert_eq!(a.mul(&b), PolyZ::from_i64_slice(&[2, 3, 1]));

// Exact division: (x^2 + 3x + 2) / (x + 1) = x + 2, no remainder.
let (q, r) = a.mul(&b).div_rem(&a).expect("x + 1 divides");
assert_eq!(q, b);
assert!(r.is_zero());
```

8.3 Resultants and discriminants

The resultant of $f, g \in \mathbb{Z}[x]$ with roots α_i, β_j (over $\overline{\mathbb{Q}}$) is

$$\operatorname{Res}(f,g) \;=\; \operatorname{lc}(f)^{\deg g}\,\operatorname{lc}(g)^{\deg f}\prod_{i,j}(\alpha_i-\beta_j) \;=\; \det S(f,g), \tag{46}$$

the determinant of the Sylvester matrix — zero exactly when f and g share a non-constant factor. `resultant` computes it by Bareiss fraction-free elimination, which keeps every intermediate entry an integer minor determinant, so no rational arithmetic appears. In the number field sieve, Res *is* the algebraic norm.

The discriminant is

$$\operatorname{disc}(f) \;=\; \frac{(-1)^{d(d-1)/2}}{\operatorname{lc}(f)}\operatorname{Res}(f,f'), \qquad d=\deg f, \tag{47}$$

zero exactly when f has a repeated factor; the division by $\operatorname{lc}(f)$ is exact over $\mathbb{Z}$.

```
// disc(x^2 + 5x + 6) = 25 - 24 = 1.
assert_eq!(PolyZ::from_i64_slice(&[6, 5, 1]).discriminant(), BigInt::from_i64(1));
// (x-1)(x-2) and (x-1)(x-3) share (x-1): resultant zero.
let u = PolyZ::from_i64_slice(&[2, -3, 1]);
let v = PolyZ::from_i64_slice(&[3, -4, 1]);
assert_eq!(u.resultant(&v), BigInt::zero());
```

8.4 Arithmetic over $\mathbb{F}_p$

`PolyMod` adds the field-division layer: `div_rem` and `rem` (schoolbook long division with the one leading-coefficient inverse hoisted out of the loop — and skipped entirely for a monic divisor; `rem` runs the same loop without quotient bookkeeping, the form the gcd descent wants), `gcd` (Euclid on remainders, result returned monic — over a field the gcd is unique only up to a unit), `make_monic`, and `mod_pow` (binary exponentiation with a polynomial reduction after every step, the primitive behind the Frobenius computations below).

```
// Over Z/7Z: (x - 1)(x - 2) and (x - 2)(x - 3) share x - 2.
let p = BigUint::from_u64(7);
let f = PolyMod::from_poly_z(&PolyZ::from_i64_slice(&[2, -3, 1]), &p);
let g = PolyMod::from_poly_z(&PolyZ::from_i64_slice(&[6, -5, 1]), &p);
let shared = PolyMod::from_poly_z(&PolyZ::from_i64_slice(&[-2, 1]), &p);
assert_eq!(f.gcd(&g), shared);
```

8.5 Factorization over $\mathbb{F}_p$

`factor` returns the monic irreducible factors with multiplicities; it composes three classical stages.

Squarefree factorization. Write $f = \prod a_i^i$ with the a_i squarefree and pairwise coprime. Because $\frac{d}{dx} a^i = i\, a^{i-1} a'$ vanishes exactly when $p \mid i$,

$$\gcd(f, f') \;=\; \prod_{p \nmid i} a_i^{i-1} \cdot \prod_{p \mid i} a_i^i, \tag{48}$$

so $f / \gcd(f, f')$ is the product of the distinct factors of multiplicity prime to p, and a gcd cascade peels them off one multiplicity at a time. What remains is a p-th power, and in $\mathbb{F}_p[x]$,

$$g(x)^p \;=\; g(x^p), \tag{49}$$

so its p-th root is read off by taking every p-th coefficient (Frobenius fixes $\mathbb{F}_p$), and the recursion continues with multiplicities scaled by p. `squarefree_factorization` exposes this stage.

Distinct-degree factorization. The key identity: over $\mathbb{F}_p$,

$$x^{p^d} - x \;=\; \prod_{\substack{h \text{ monic irreducible} \\ \deg h \mid d}} h(x), \tag{50}$$

so $\gcd\!\left(f,\; x^{p^d} - x\right)$ extracts from a squarefree f the product of its irreducible factors of degree dividing d. Computing $x^{p^d} \bmod f$ is d Frobenius steps by `mod_pow`; sweeping $d = 1, 2, \ldots$ splits f into blocks of equal-degree factors.

Equal-degree splitting (Cantor–Zassenhaus). A block f of $k > 1$ irreducible factors, all of degree d, splits probabilistically: for a uniform random h with $\deg h < \deg f$ and odd p,

$$\gcd\!\left(h^{(p^d-1)/2} - 1,\; f\right) \tag{51}$$

is a proper factor with probability at least $\frac{1}{2}$ per pair of factors, because rem each irreducible factor h lands in $\mathbb{F}_{p^d}^{\times}$ and $h^{(p^d-1)/2} = \pm 1$ independently, each sign with probability near $\frac{1}{2}$. For $p = 2$, where $(p^d - 1)/2$ is not an integer, the splitter is the trace map $T(h) = h + h^2 + h^4 + \cdots + h^{2^{d-1}}$ instead, which takes each factor to $\mathbb{F}_2$ with the same independent coin-flip behaviour. The routine is Las Vegas — retries never return a wrong split — and caps consecutive fruitless draws so a broken generator fails loudly rather than spinning.

`is_irreducible` runs the distinct-degree sieve as a test; `roots(rng)` extracts the linear factors of $\gcd(f, x^p - x)$ and returns the residues where f vanishes. Factoring and root-finding take an `RandomSource` (§11).

```
struct Lcg(u64);
impl RandomSource for Lcg {
    fn fill_bytes(&mut self, d: &mut [u8]) {
        for b in d {
            self.0 = self.0.wrapping_mul(6364136223846793005).wrapping_add(1);
```

```
            *b = (self.0 >> 33) as u8;
        }
    }
}
let p = BigUint::from_u64(7);
let mut rng = Lcg(0x1234_5678);

// x^2 - 1 = (x - 1)(x + 1) mod 7 - two roots.
let f = PolyMod::from_poly_z(&PolyZ::from_i64_slice(&[-1, 0, 1]), &p);
let mut roots = f.roots(&mut rng);
roots.sort();
assert_eq!(roots, vec![BigUint::from_u64(1), BigUint::from_u64(6)]);

// x^2 + 1 is irreducible mod 7 (-1 is a non-residue), so no roots.
let g = PolyMod::from_poly_z(&PolyZ::from_i64_slice(&[1, 0, 1]), &p);
assert!(g.is_irreducible());
assert!(g.roots(&mut rng).is_empty());

// factor returns monic irreducibles with multiplicities:
// x^3 + x = x * (x^2 + 1) over F_7.
let h = PolyMod::from_poly_z(&PolyZ::from_i64_slice(&[0, 1, 0, 1]), &p);
let mut fs = h.factor(&mut rng);
fs.sort_by_key(|(fac, _)| fac.degree());
assert_eq!(fs.len(), 2);
```

8.6 Quotient rings, lifting, and base expansions

Let $f \in \mathbb{Z}[x]$ be monic. Division by f cannot fail over $\mathbb{Z}$, its leading coefficient being a unit, so `rem_monic` returns a value where `div_rem` must return an `Option`, and

$$\mathbb{Z}[x] \longrightarrow \mathbb{Z}[x]/(f), \qquad g \longmapsto g \bmod f$$

is a ring homomorphism. Monicity also makes each step cheap: the quotient coefficient *is* the remainder's leading coefficient, so a step is one multiply–subtract across the divisor's window with no coefficient division anywhere, and the quotient is never formed. A divisor with $\mathrm{lc} = -1$ generates the same ideal; negate it and the remainder is unchanged.

`product_mod_monic` forms $\prod_i g_i$ in $\mathbb{Z}[x]/(f)$ by a product tree. A fold multiplies a running product, whose coefficients grow with every term, by a fresh small one, so the work at step k is proportional to k and the total is quadratic in the number of factors; pairing keeps both operands the same size at every level — the shape `mul`'s Karatsuba and Toom kernels want — and the depth is logarithmic. Reducing at every level rather than once at the end is sound precisely because reduction is a homomorphism, and it is what holds every intermediate to degree $\deg f$ instead of letting the degree grow with the number of factors.

`balanced_base_expansion` writes

$$n = \sum_{k=0}^{d} c_k m^k, \qquad |c_k| \le m/2 \quad (k < d),$$

by the recurrence $c_k = \mathrm{symrem}(r_k, m)$ and $r_{k+1} = (r_k - c_k)/m$, the division being exact because $r_k \equiv c_k \pmod m$. Only the first d digits are reduced; c_d carries whatever remains, so the identity

is exact for *every* requested d and choosing d too small loses no information. Against the ordinary expansion with digits in $[0, m)$ this halves $\max_k |c_k|$ at no cost, which is why polynomial selection for the number-field sieve uses it: there $f(m) = n$ by construction and the norms one needs small are governed by the coefficient bound.

`homogeneous_substitution` evaluates the homogenization

$$F(X, Y) = Y^d f(X/Y) = \sum_{k=0}^{d} c_k X^k Y^{d-k}$$

at a pair of polynomials (a, b) — the unique degree-d form in two variables restricting to f on $Y = 1$. Where `evaluate` answers "what is f at a point", this answers "what does f become under a projective change of variables", and the two agree at $b = 1$. It is the form to use whenever the argument is a ratio: an algebraic norm $b^d f(a/b)$ is this at constants, and composing a linear change of coordinates into f is this at linear a and b. Both power ladders are built once, so the cost is $2d$ multiplications to build and $d + 1$ to combine rather than repeated powering, and zero coefficients are skipped.

`roots_mod_prime_power` returns every root of f rem p^e by Hensel lifting. Taylor truncated after one term is exact one level up,

$$f(r + tp^k) \equiv f(r) + t\, p^k f'(r) \pmod{p^{k+1}},$$

and writing $f(r) = s\, p^k$ — legitimate because r is a root rem p^k — the congruence is linear in t. Where $f'(r) \not\equiv 0 \pmod p$ the coefficient is invertible and there is exactly one lift, $t \equiv -s\, f'(r)^{-1}$; where $f'(r) \equiv 0$ the t term vanishes and the congruence no longer mentions t, so either $p^{k+1} \mid f(r)$ and all p lifts are roots or none is. The simple case is Hensel's lemma (Cohen, §1.6); the branching case is what a general root-finder must add to it, and it is why the number of roots rem p^e is not in general the number rem p — the multiple roots, those lying over primes dividing $\operatorname{disc} f$, are where the tree widens.

Two lifts move between $\mathbb{Z}/m\mathbb{Z}$ and $\mathbb{Z}$. `symmetric_lift` takes each coefficient to its representative of least absolute value in $(-m/2, m/2]$: `coefficients` exposes the canonical representatives in $[0, m)$, which is the right normal form for arithmetic and the wrong one for size, since a class near m is a small negative number in disguise. A modulus wider than twice the height therefore returns the true integer polynomial exactly. `with_modulus` re-reads the same representatives at another modulus, and the two divisibility directions behave differently. Narrowing $(m' \mid m)$ is the projection $\mathbb{Z}/m\mathbb{Z} \to \mathbb{Z}/m'\mathbb{Z}$, a ring homomorphism, so sums and products agree before and after. Widening $(m \mid m')$ is that projection's canonical section, and a section between rings of different size is never additive — at $m = 7$, $m' = 49$, $5 + 4$ widens to 2 but widens-then-adds to 9. Only equality survives. Widening is nonetheless the operation wanted: it is the seeding step of a Newton lift.

```
struct Lcg2(u64);
impl RandomSource for Lcg2 {
    fn fill_bytes(&mut self, d: &mut [u8]) {
        for b in d {
            self.0 = self.0.wrapping_mul(6364136223846793005).wrapping_add(1);
            *b = (self.0 >> 33) as u8;
        }
    }
}
```

```
let mut rng = Lcg2(0x2024_1111);

// Reduction rem the monic x^2 + 1 is a ring homomorphism.
let f = PolyZ::from_i64_slice(&[1, 0, 1]);
let a = PolyZ::from_i64_slice(&[3, 2, 5]); // 5x^2 + 2x + 3
let b = PolyZ::from_i64_slice(&[1, 7]);    // 7x + 1
assert_eq!(
    a.mul(&b).rem_monic(&f),
    a.rem_monic(&f).mul(&b.rem_monic(&f)).rem_monic(&f)
);

// The product tree in Z[x]/(f) agrees with the fold, reduced once.
let x = PolyZ::from_i64_slice(&[0, 1]);
let factors = [a.clone(), b.clone(), x.clone()];
assert_eq!(
    PolyZ::product_mod_monic(&factors, &f),
    a.mul(&b).mul(&x).rem_monic(&f)
);

// Balanced base-1000 expansion of 1234567890: digits in (-500, 500].
let n = BigInt::from_i64(1_234_567_890);
let base = BigUint::from_u64(1_000);
let g = PolyZ::balanced_base_expansion(&n, &base, 3);
assert_eq!(g, PolyZ::from_i64_slice(&[-110, -432, 235, 1]));
assert_eq!(g.evaluate(&BigInt::from_i64(1_000)), n);

// Homogenization of x^2 + 1 at (2x, 3): 3^2 + (2x)^2 = 4x^2 + 9.
assert_eq!(
    f.homogeneous_substitution(
        &PolyZ::from_i64_slice(&[0, 2]),
        &PolyZ::from_i64_slice(&[3])
    ),
    PolyZ::from_i64_slice(&[9, 0, 4])
);

// Square roots of 2 rem 7^3 = 343, lifted from +-3 rem 7.
let sqrt2 = PolyZ::from_i64_slice(&[-2, 0, 1])
    .roots_mod_prime_power(&BigUint::from_u64(7), 3, &mut rng);
assert_eq!(sqrt2, vec![BigUint::from_u64(108), BigUint::from_u64(235)]);

// A symmetric lift recovers the integer polynomial it came from.
let wide = BigUint::from_u64(2).pow_u64(96);
assert_eq!(PolyMod::from_poly_z(&a, &wide).symmetric_lift(), a);
```

8.7 Real roots

`PolyZ::real_roots` returns the real roots ascending, each repeated according to its multiplicity, as `Result<Vec<f64>, RealRootError>`.

Multiplicities are settled exactly in $\mathbb{Z}$ before any floating point is used. A bisection locates a root only where the polynomial changes sign, so a root of even multiplicity — where f touches zero and turns back — is invisible to it, and two nearby simple roots cannot be told from one

double root at `f64` precision. The squarefree decomposition $f = s_1 s_2^2 s_3^3 \cdots$, obtained by repeated exact gcd with f', makes each s_k squarefree, so its real roots are simple and bracketing finds all of them; each is then emitted k times.

Each squarefree factor is bracketed and bisected. The real roots of its derivative, found by the same procedure recursively, split the line into intervals on which the factor is monotone, so each holds at most one root (`simple_real_roots`). The outer ends are Cauchy's bound $1 + \max_i |c_i/c_d|$, rounded outward at every step — each ratio an exact integer ceiling, the sum stepped one `f64` up — so the bracket can only be too wide (`cauchy_bound`). Every sign the bisection consults is decided exactly: an `f64` is a dyadic rational $m \cdot 2^e$, so $2^{kd} f(m2^e)$ with $k = \max(0, -e)$ is an integer that `BigInt` sums without rounding (`sign_at`). The bracket is therefore a genuine sign change at every step, and `bisect_f64` runs until the midpoint meets an endpoint — the end of the format's resolution, not of a step budget — returning a value one `f64` from a true root. Floating point carries only the split points and the answer; every decision is made in $\mathbb{Z}$.

An empty `Ok` is an answer — no real roots, the ordinary case for even degree. The refusals are distinct: `ZeroPolynomial`, where every real number is a root, and `CoefficientOutOfRange`, where a coefficient does not fit `f64` and the polynomial cannot be evaluated at all.

```
// (x-1)(x-2)(x-3)
let f = PolyZ::from_i64_slice(&[-6, 11, -6, 1]);
let roots = f.real_roots().expect("finite coefficients");
assert_eq!(roots.len(), 3);
assert!((roots[0] - 1.0).abs() < 1e-9);

// (x-2)^2: a double root, returned twice.
let squared = PolyZ::from_i64_slice(&[4, -4, 1]);
assert_eq!(squared.real_roots().expect("finite").len(), 2);

// No real roots is an answer; the zero polynomial is a refusal.
assert_eq!(PolyZ::from_i64_slice(&[1, 0, 1]).real_roots(), Ok(Vec::new()));
assert_eq!(PolyZ::zero().real_roots(), Err(RealRootError::ZeroPolynomial));
```

9 Lattice reduction: `lll_reduce`

`lll_reduce(basis)` applies the Lenstra–Lenstra–Lovász algorithm to an ordered lattice basis — the rows $b_1, \ldots, b_n$ of a `&mut [Vec<BigInt>]` — replacing it in place with a short, nearly orthogonal basis of the same lattice. The rows must be linearly independent.

With Gram–Schmidt vectors and coefficients

$$b_i^* \;=\; b_i - \sum_{j<i} \mu_{ij}\, b_j^*, \qquad \mu_{ij} \;=\; \frac{\langle b_i, b_j^* \rangle}{\langle b_j^*, b_j^* \rangle}, \tag{52}$$

a basis is *LLL-reduced* with parameter δ when

$$|\mu_{ij}| \le \tfrac{1}{2} \;\; (j < i) \qquad \text{and} \qquad (\delta - \mu_{i,i-1}^2)\,\|b_{i-1}^*\|^2 \;\le\; \|b_i^*\|^2 \quad \text{(Lovász condition).} \tag{53}$$

For $\delta = \frac{3}{4}$ (the default; `lll_reduce_delta` takes any fraction in $(\frac{1}{4}, 1)$) the first vector of a reduced basis satisfies

$$\|b_1\| \;\le\; 2^{(n-1)/4} (\det L)^{1/n}, \qquad \|b_1\| \;\le\; 2^{(n-1)/2}\, \lambda_1(L), \tag{54}$$

where $\lambda_1(L)$ is the length of a shortest non-zero lattice vector. The implementation is the *integral* variant (Cohen, Algorithm 2.6.3): it carries the Gram–Schmidt data as exact integers $d_j = \prod_{i \le j} \|b_i^*\|^2$ and $\lambda_{ij} = d_j\, \mu_{ij}$, so no rational or floating-point arithmetic appears and the output is exact. An independent rational-LLL oracle in the test suite pins its behavior.

```
let row = |xs: &[i64]| xs.iter().map(|&x| BigInt::from_i64(x)).collect::<Vec<_>>();
// A badly skewed basis for a 2-D lattice.
let mut basis = vec![row(&[201, 37]), row(&[1648, 297])];
lll_reduce(&mut basis);
// Reduction returns short vectors spanning the same lattice.
assert_eq!(basis, vec![row(&[1, 32]), row(&[40, 1])]);

// An explicit Lovasz parameter, as the fraction 99/100.
let mut basis = vec![row(&[201, 37]), row(&[1648, 297])];
lll_reduce_delta(&mut basis, 99, 100);
assert_eq!(basis, vec![row(&[1, 32]), row(&[40, 1])]);
```

9.1 Two dimensions under a diagonal form

In two dimensions reduction is not a heuristic. Lagrange–Gauss reduction solves the shortest-vector problem outright: `gauss_reduce_weighted` returns a shortest non-zero vector of the lattice first and a shortest vector independent of it second, under the diagonal form

$$\|(x, y)\|^2 \;=\; (w_0 x)^2 + (w_1 y)^2. \tag{55}$$

Each round replaces the shorter vector with a strictly shorter one, so the squared norms strictly decrease in the positive integers and the loop runs $O(\log)$ times; no iteration cap is imposed.

For a skewed metric $(x/\sqrt{s})^2 + (y\sqrt{s})^2$ with rational $s = p/q$, multiply the form through by pq: a positive scale factor changes neither the comparisons nor the rounding, and this clears the square roots, giving $(qx)^2 + (py)^2$, so $w = (q, p)$. An integer skew is the case $q = 1$. A skew that is not rational must be approximated first, and the reduction is then exact under the approximating form rather than the intended one — the arithmetic is exact, the form only as faithful as p/q, and the denominator is paid for out of range, since norms grow as the square of the weights. Within the form given there is no accuracy cliff, which is the difference from a floating-point metric that loses the ordering once the weighted coordinates pass 2^{53}. A dependent pair is `Err(ReductionError::DependentBasis)` and arithmetic past `i128` is `Err(ReductionError::OutOfRange)`; the weights are `NonZeroU64`, so a non-positive weight cannot be written. The rounding step forms $2\langle u, v\rangle + \|u\|^2$, so every weighted squared norm S the reduction visits needs $3S \le$ `i128::MAX`, that is $S \le (2^{127} - 1)/3 \approx 2^{125.4}$.

```
// Weights are NonZeroU64, so a non-positive weight cannot be written.
let nz = |n: u64| NonZeroU64::new(n).expect("literal is non-zero");

// A skew-12 lattice, reduced under the matching diagonal form.
let basis = [[1024i128, 0], [37, 1]];
let reduced = gauss_reduce_weighted(basis, [nz(1), nz(12)]).expect("valid");
let det = |b: [[i128; 2]; 2]| b[0][0] * b[1][1] - b[0][1] * b[1][0];
assert_eq!(det(reduced).abs(), det(basis).abs());

// Weights reorder what counts as short.
```

```
let square = [[1i128, 0], [0, 1]];
assert_eq!(gauss_reduce_weighted(square, [nz(100), nz(1)]), Ok([[0, 1], [1, 0]]));
assert_eq!(gauss_reduce_weighted(square, [nz(1), nz(100)]), Ok([[1, 0], [0, 1]]));

// A dependent pair is a typed error rather than a panic.
assert_eq!(gauss_reduce_weighted([[2, 4], [1, 2]], [nz(1), nz(1)]),
           Err(ReductionError::DependentBasis));
```

10 Linear algebra over a prime field: `gfp`

An index-calculus discrete logarithm ends in a homogeneous system $Mx = 0$ over $\mathbb{F}_\ell$, where ℓ is the order of the group: the rows of M are the exponent vectors of the relations and x is the vector of logarithms. The entries of M are small integers — overwhelmingly ± 1 — while the entries of x are residues hundreds of bits wide, and `gfp` is built around that asymmetry. `SparseMatrix::new` takes each row as `(column, coefficient)` pairs and keeps the ± 1 entries apart from the rest, so a row's product is a run of limb additions and subtractions reduced once at the end; the matrix must be square, since the solver forms powers of it.

`kernel_vector` is Wiedemann's algorithm (Wiedemann, *Solving sparse linear equations over finite fields*, IEEE Trans. Inform. Theory 32 (1986)). For random u, v the scalars $a_k = u^{\mathsf{T}} M^k v$ obey a linear recurrence whose minimal polynomial f divides that of M, and $2n$ of them determine it (Berlekamp–Massey; `minimal_polynomial` returns f low coefficient first and monic). Writing $f = x^m g$ with $g(0) \neq 0$, the vector $g(M)\,v$ is killed by some power of M, and the last non-zero iterate before that is in the kernel. The result is *some* kernel element: a caller that wants one particular line makes the kernel one-dimensional first.

The outcome is a `Kernel`. `Vector` carries the answer. `Inconclusive` is a failed draw — about n/ℓ of them are — and the response is another draw. `NoKernel` is a statement about the matrix: zero is not a root of the recurrence, so no draw will find a kernel. Telling the two apart is what lets a caller retry the right thing.

```
/// A deterministic source for the example. Not a CSPRNG.
struct Draws(u64);

impl RandomSource for Draws {
    fn fill_bytes(&mut self, dest: &mut [u8]) {
        for chunk in dest.chunks_mut(8) {
            self.0 ^= self.0 << 13;
            self.0 ^= self.0 >> 7;
            self.0 ^= self.0 << 17;
            let word = self.0.to_le_bytes();
            chunk.copy_from_slice(&word[..chunk.len()]);
        }
    }
}

// The field GF(2^61 - 1).
let field = Field::new(BigUint::from_u64((1 << 61) - 1)).expect("a modulus above one
    ");

// A 3 x 3 matrix whose third row is the sum of the first two, so its
```

```
// kernel is the line through (1, 1, 1).
let matrix = SparseMatrix::new(
    3,
    vec![
        vec![(0, 1), (1, 1), (2, -2)],
        vec![(0, 1), (1, -1)],
        vec![(0, 2), (2, -2)],
    ],
)
.expect("square, in range, no repeated column");
assert_eq!(matrix.nonzeros(), 7);

// A draw can fail with probability about n/l; this one does not.
let mut rng = Draws(7);
let Kernel::Vector(x) = kernel_vector(&matrix, &field, &mut rng) else {
    panic!("the draw failed; a real caller draws again");
};
assert!(matrix.multiply(&field, &x).iter().all(BigUint::is_zero));
assert_eq!(x[0], x[1]);
assert_eq!(x[1], x[2]);
assert!(!x[0].is_zero());

// An invertible matrix is named as such, not reported as bad luck.
let identity = SparseMatrix::new(2, vec![vec![(0, 1)], vec![(1, 1)]]).expect("valid"
    );
assert_eq!(kernel_vector(&identity, &field, &mut rng), Kernel::NoKernel);

// The recurrence behind it: Berlekamp-Massey on the Fibonacci numbers
// finds x^2 - x - 1, returned low coefficient first and monic.
let mut fib = vec![BigUint::zero(), BigUint::one()];
for i in 2..8 {
    let next = field.add(&fib[i - 1], &fib[i - 2]);
    fib.push(next);
}
let minus_one = field.sub(&BigUint::zero(), &BigUint::one());
assert_eq!(
    minimal_polynomial(&field, &fib),
    Some(vec![minus_one.clone(), minus_one, BigUint::one()])
);
```

11 Random sampling

rump chooses no entropy source. Every sampler is driven by a caller-supplied generator implementing the one-method trait

```
pub trait RandomSource {
    fn fill_bytes(&mut self, dest: &mut [u8]);
}
```

and the output is exactly as good as that source — cryptographic callers must supply a CSPRNG.

Every sampler is a *rejection* sampler, which buys exact uniformity: reducing a wide draw

rem the bound would over-represent the low residues whenever the bound is not a power of two. `random_below` draws uniformly from the enclosing power-of-two range $[0, 2^b)$, b the bound's bit length, and rejects draws at or above the bound; since the bound is at least 2^{b-1}, each draw is accepted with probability at least $\frac{1}{2}$ and the expected number of draws is at most 2. `random_nonzero_below` additionally rejects zero; `random_coprime_below(rng, n, m)` keeps the first draw coprime to m — pass n as both bound and modulus to draw a uniform unit of $(\mathbb{Z}/n\mathbb{Z})^\times$. `random_probable_prime(rng, bits)` draws odd candidates of exactly the requested width (top and low bits forced) and screens them with `is_probable_prime`; by the prime number theorem the expected number of rounds is about $0.35 \cdot \text{bits}$.

Degenerate *arguments* return `None` (an empty range, a zero coprimality modulus, a width below 2). For a degenerate *generator*, each sampler carries a stall guard that panics rather than loop forever on the sources it can soundly detect, every guard sized so a working generator trips it with probability at most $e^{-111} \approx 2^{-160}$, the loosest of the individual bounds (each bullet gives its own). Their coverage differs, and the one gap is stated rather than papered over:

- `random_below` and `random_nonzero_below` accept each draw with probability at least $\frac{1}{2}$ regardless of arguments, so they bound consecutive *rejections* at 256 (survival probability at most 2^{-256}) and catch every degenerate source.
- `random_probable_prime`'s acceptance density is a function of the width alone, so its rejection bound scales with the width: $64 \cdot \text{bits}$ fruitless rounds, which a working generator survives with probability $(1 - \frac{2}{\text{bits} \cdot \ln 2})^{64 \cdot \text{bits}} \approx e^{-185}$ by the asymptotic prime density, and below e^{-111} unconditionally at every width: for bits ≥ 6 from the lower bound $\pi(2x) - \pi(x) > \frac{3}{5}\, x / \ln x$ (Rosser–Schoenfeld 1962), and for the four smaller widths by exhaustive enumeration (widths 2 and 3 contain no composite; the worst case is width 4, density $\frac{1}{2}$ over 256 rounds, survival 2^{-256}) — so it, too, catches every degenerate source. A *constant* generator additionally fails fast: a candidate equal to the previous rejected one is known composite, skips the Miller–Rabin re-screen, and trips a 256-repeat assertion — one screen and 256 comparisons, not hours of full-width exponentiation at large widths.
- `random_coprime_below` is the one sampler where no *usable* rejection count exists. A primorial modulus legitimately leaves 1 as the only unit below the bound, so acceptance can be as thin as $1/(\mathit{upper} - 1)$; the only sound count therefore scales with the bound itself ($\Theta(\mathit{upper})$ draws), unreachable for a multiprecision bound. Its guard instead detects a *pinned* generator (the same rejected candidate 256 times running, probability at most 2^{-256} for a working source). A degenerate source that cycles among several rejected values is statistically indistinguishable from an unlucky legitimate run against a sparse unit set, does not trip the guard, and remains the caller's to avoid.

```
/// xorshift64: deterministic and compact. Fine for a manual; NOT a CSPRNG.
struct XorShift64(u64);

impl RandomSource for XorShift64 {
    fn fill_bytes(&mut self, dest: &mut [u8]) {
        for chunk in dest.chunks_mut(8) {
            self.0 ^= self.0 << 13;
            self.0 ^= self.0 >> 7;
            self.0 ^= self.0 << 17;
```

```
            let word = self.0.to_le_bytes();
            chunk.copy_from_slice(&word[..chunk.len()]);
        }
    }
}

let mut rng = XorShift64(0x1234_5678_9abc_def0);
let bound = BigUint::from_u64(1_000_003);

let draw = random_below(&mut rng, &bound).expect("bound is non-zero");
assert!(draw < bound);

let nonzero = random_nonzero_below(&mut rng, &bound).expect("bound exceeds one");
assert!(!nonzero.is_zero() && nonzero < bound);

let unit = random_coprime_below(&mut rng, &bound, &BigUint::from_u64(30_030))
    .expect("units exist below the bound");
assert_eq!(gcd(&unit, &BigUint::from_u64(30_030)), BigUint::one());

let prime = random_probable_prime(&mut rng, 64).expect("width of at least 2");
assert_eq!(prime.bits(), 64);
assert!(is_probable_prime(&prime));
```

12 Panics

The unsigned types treat impossible requests as programming errors, not recoverable conditions. Fallible *mathematics* — a missing inverse, a non-residue, an even Montgomery modulus, non-coprime CRT moduli, a residue from another context — returns `Option` or `Result` instead.

Call	Panics when
`sub` / `sub_into` / `-=`	the result would be negative
`div_rem` / `div_rem_u64` / `rem` / `rem_u64`	the divisor or modulus is zero
`mod_mul` / `mod_pow` / `mod_add` / `mod_sub` / `mod_neg`	the modulus is zero
`ln_approx`	the value is zero
`digit_count`	the radix is below 2
`mod_inverse_u64`	the modulus is zero
`nth_root_floor`	$k = 0$
`mod_sqrt_prime_power`	$e = 0$ or $p < 2$
`valuation` / `remove_factor`	$n = 0$ or $p < 2$
`remainder_tree`	a leaf is zero
`rational_reconstruct_bounded`	$2ND \geq m$ (the uniqueness contract)
`to_be_bytes_padded` / `to_le_bytes_padded`	the value exceeds the requested width
`from_str_radix` / `to_str_radix`	the radix is outside $[2, 36]$
`Gf2m::half_trace`	the field degree is even
`Gf2m::trace`	a reducible modulus makes the Frobenius sum leave GF(2)
`PolyZ` / `PolyMod` division	the divisor is the zero polynomial
`PolyMod::new` / `zero` / `from_poly_z`	the modulus is below 2
`PolyMod` (mixed moduli)	the two operands carry different moduli
`PolyMod` division / `gcd` / `factor`	a non-invertible pivot (composite modulus)
`squarefree_factorization` / `factor`	the polynomial is constant or zero
`factor` / `roots`	the supplied `RandomSource` makes no progress (the equal-degree splitter's stall guard)
`rem_monic` / `product_mod_monic`	the divisor is zero or is not monic
`balanced_base_expansion`	the base is below 2
`roots_mod_prime_power`	a zero exponent, a base below 2, a polynomial that is zero or has every coefficient divisible by p^e, or a lift level or answer wider than `MAX_ENUMERATED_ROOTS` (2^{20} candidates)
`lll_reduce` / `lll_reduce_delta`	dependent rows, ragged or zero-length rows; the `_delta` form also on $\delta \notin (\frac{1}{4}, 1)$ or a zero denominator
samplers (§11)	the generator trips a stall guard (see §11 for what each guard can and cannot detect)

References

- D. E. Knuth, *The Art of Computer Programming*, vol. 2: *Seminumerical Algorithms*, 3rd ed. — Algorithms M, D, L; §4.3.3 (Karatsuba, Toom–Cook); §4.6.3 (exponentiation).
- P. L. Montgomery, *Modular multiplication without trial division*, Math. Comp. 44 (1985).

- S. R. Dussé and B. S. Kaliski, *A cryptographic library for the Motorola DSP56000*, EUROCRYPT '90 — the word-level n'_0.
- A. J. Menezes, P. C. van Oorschot, S. A. Vanstone, *Handbook of Applied Cryptography* — Algorithms 14.42, 14.82; §3.5.1.
- N. Möller, *On Schönhage's algorithm and subquadratic integer GCD computation*, Math. Comp. 77 (2008); the Jacobi threading as realized in GMP's `mpn_hgcd_jacobi`.
- R. P. Brent and P. Zimmermann, *Modern Computer Arithmetic*, Cambridge, 2010 — §4.2.2 (Newton's reciprocal), §2.4 (Barrett's division).
- R. P. Brent and P. Zimmermann, *An $O(M(n)\log n)$ algorithm for the Jacobi symbol*, ANTS-IX, LNCS 6197 (2010).
- N. Möller and T. Granlund, *Improved division by invariant integers*, IEEE Trans. Computers 60 (2011) — Algorithm 4.
- J. Sorenson and J. Webster, *Strong pseudoprimes to twelve prime bases*, Math. Comp. 86 (2017).
- H. Cohen, *A Course in Computational Algebraic Number Theory* — Algorithms 1.7.1, 2.6.3; §3.3–3.4.
- R. Baillie and S. S. Wagstaff, *Lucas pseudoprimes*, Math. Comp. 35 (1980).
- National Institute of Standards and Technology, *Digital Signature Standard (DSS)*, FIPS PUB 186-4 (2013) — Appendix C.3.3.
- M. O. Rabin, *Probabilistic algorithms in finite fields*, SIAM J. Comput. 9 (1980).
- D. Hankerson, A. Menezes, S. Vanstone, *Guide to Elliptic Curve Cryptography* — Algorithms 2.36, 2.48; §2.3.5.
- D. J. Bernstein, *How to find smooth parts of integers*, 2004.
- A. K. Lenstra, H. W. Lenstra, L. Lovász, *Factoring polynomials with rational coefficients*, Math. Ann. 261 (1982).
- J. von zur Gathen and J. Gerhard, *Modern Computer Algebra*, 3rd ed. — §5.10, §14 (Cantor–Zassenhaus).

The crate's own `CITATIONS.md` maps each routine to its primary source at finer grain.